Using the Open Source ASN.1 Compiler

Documentation for asn1c version
v1.5.3-30-gcb8c9e8d-2026-10-12

Lev Walkin <vlm@lionet.info>
Mouse <5923577+mouse07410@users.noreply.github.com>

October 12, 2026

Contents

1 Quick start examples **4**
- 1.1 A “Rectangle” converter and debugger . . . 4
- 1.2 A “Rectangle” Encoder . . . 5
- 1.3 A “Rectangle” Decoder . . . 7
- 1.4 Adding constraints to a “Rectangle” . . . 9

2 ASN.1 Compiler **10**
- 2.1 The asn1c compiler tool . . . 10
- 2.2 Compiler output . . . 12
- 2.3 Command line options . . . 13

3 API reference **18**
- 3.1 ASN_STRUCT_FREE() macro . . . 19
- 3.2 ASN_STRUCT_RESET() macro . . . 20
- 3.3 asn_check_constraints() . . . 21
- 3.4 asn_decode() . . . 23
- 3.5 asn_encode() . . . 26
- 3.6 asn_encode_to_buffer() . . . 28
- 3.7 asn_encode_to_new_buffer() . . . 30
- 3.8 asn_fprint() . . . 32
- 3.9 asn_random_fill() . . . 33
- 3.10 ber_decode() . . . 34
- 3.11 der_encode . . . 36
- 3.12 der_encode_to_buffer() . . . 38
- 3.13 oer_decode() . . . 40
- 3.14 oer_encode() . . . 42
- 3.15 oer_encode_to_buffer() . . . 44

3.16 uper_decode() . . . 46
3.17 uper_decode_complete() . . . 48
3.18 uper_encode() . . . 50
3.19 uper_encode_to_buffer() . . . 51
3.20 uper_encode_to_new_buffer() . . . 52
3.21 xer_decode() . . . 53
3.22 xer_encode() . . . 55
3.23 xer_fprint() . . . 57

4 CBOR API usage examples **58**
4.1 cbor_decode() . . . 59
4.2 cbor_encode() . . . 60
4.3 CBOR Tags . . . 61

5 API usage examples **64**
5.1 Generic encoders and decoders . . . 64
5.2 Encoding CER . . . 65
5.3 Generating Rust code . . . 67
5.4 Decoding BER . . . 68
5.5 Encoding DER . . . 70
5.6 Encoding XER . . . 71
5.7 Decoding XER . . . 72
5.8 Validating the target structure . . . 73
5.9 Printing the target structure . . . 73
5.10 Freeing the target structure . . . 73

6 Advanced Features **75**
6.1 Unknown extension decoding . . . 75
6.2 ENCODING-CONTROL Directives . . . 76
6.3 Circular Dependency Handling . . . 79
6.4 Information Object Class Support . . . 82
6.5 Using the `-fprefix` Option . . . 86

7 Abstract Syntax Notation: ASN.1 **89**
7.1 Some of the ASN.1 Basic Types . . . 90
7.1.1 The BOOLEAN type . . . 90
7.1.2 The INTEGER type . . . 90

7.1.3 The ENUMERATED type . . . 91
7.1.4 The OCTET STRING type . . . 92
7.1.5 The OBJECT IDENTIFIER type . . . 92
7.1.6 The RELATIVE-OID type . . . 93
7.2 Some of the ASN.1 String Types . . . 93
7.2.1 The IA5String type . . . 93
7.2.2 The UTF8String type . . . 93
7.2.3 The NumericString type . . . 93
7.2.4 The PrintableString type . . . 93
7.2.5 The VisibleString type . . . 93
7.3 ASN.1 Constructed Types . . . 94
7.3.1 The SEQUENCE type . . . 94
7.3.2 The SET type . . . 94
7.3.3 The CHOICE type . . . 94
7.3.4 The SEQUENCE OF type . . . 95
7.3.5 The SET OF type . . . 95

Chapter 1

Quick start examples

1.1 A "Rectangle" converter and debugger

One of the most common need is to create some sort of analysis tool for the existing ASN.1 data files. Let's build a converter for existing Rectangle binary files between BER, OER, PER, and XER (XML).

1. Create a file named **rectangle.asn** with the following contents:

```
RectangleModule DEFINITIONS ::= BEGIN

Rectangle ::= SEQUENCE {
    height  INTEGER,
    width   INTEGER
}

-- ENCODING-CONTROL directives can be added to customize XER
   encoding.
-- See section 6.2 on page 76 for details.

END
```

2. Compile it into the set of .c and .h files using `asn1c` compiler:

```
asn1c -no-gen-example rectangle.asn
```

3. Create the converter and dumper:

```
make -f converter-example.mk
```

4. Done. The binary file converter is ready:

```
./converter-example -h
```

1.2 A "Rectangle" Encoder

This example will help you create a simple BER and XER encoder of a "Rectangle" type used throughout this document.

1. Create a file named **rectangle.asn** with the following contents:

```
RectangleModule DEFINITIONS ::= BEGIN

Rectangle ::= SEQUENCE {
    height  INTEGER,
    width   INTEGER
}

END
```

2. Compile it into the set of .c and .h files using asn1c compiler [ASN1C]:

```
asn1c -no-gen-example rectangle.asn
```

3. Alternatively, use the Online ASN.1 compiler [AONL] by uploading the **rectangle.asn** file into the Web form and unpacking the produced archive on your computer.

4. By this time, you should have gotten multiple files in the current directory, including the **Rectangle.c** and **Rectangle.h**.

5. Create a main() routine which creates the Rectangle_t structure in memory and encodes it using BER and XER encoding rules. Let's name the file **main.c**:

```
#include <stdio.h>
#include <sys/types.h>
#include <Rectangle.h>   /* Rectangle ASN.1 type */

/* Write the encoded output into some FILE stream. */
static int write_out(const void *buffer, size_t size, void *app_key) {
    FILE *out_fp = app_key;
```

```
    size_t wrote = fwrite(buffer, 1, size, out_fp);
    return (wrote == size) ? 0 : -1;
}

int main(int ac, char **av) {
    Rectangle_t *rectangle; /* Type to encode        */
    asn_enc_rval_t ec;      /* Encoder return value  */

    /* Allocate the Rectangle_t */
    rectangle = calloc(1, sizeof(Rectangle_t)); /* must initialize to zero, not
        malloc! */
    if(!rectangle) {
        perror("calloc() failed");
        exit(1);
    }

    /* Initialize the Rectangle members */
    rectangle->height = 42;  /* any random value */
    rectangle->width  = 23;  /* any random value */

    /* BER encode the data if filename is given */
    if(ac < 2) {
        fprintf(stderr, "Specify filename for BER output\n");
    } else {
        const char *filename = av[1];
        FILE *fp = fopen(filename, "wb");   /* for BER output */

        if(!fp) {
          perror(filename);
          exit(1);
        }

        /* Encode the Rectangle type as BER (DER) */
        ec = der_encode(&asn_DEF_Rectangle, rectangle, write_out, fp);
        fclose(fp);
        if(ec.encoded == -1) {
          fprintf(stderr, "Could not encode Rectangle (at %s)\n",
              ec.failed_type ? ec.failed_type->name : "unknown");
          exit(1);
        } else {
          fprintf(stderr, "Created %s with BER encoded Rectangle\n", filename);
        }
    }

    /* Also print the constructed Rectangle XER encoded (XML) */
    xer_fprint(stdout, &asn_DEF_Rectangle, rectangle);

    return 0; /* Encoding finished successfully */
}
```

6. Compile all files together using C compiler (varies by platform):

   `cc -I. -o` ***`rencode`*** `*.c`

7. Done. You have just created the BER and XER encoder of a Rectangle type, named **rencode**!

1.3 A "Rectangle" Decoder

This example will help you to create a simple BER decoder of a simple "Rectangle" type used throughout this document.

1. Create a file named **rectangle.asn** with the following contents:

```
RectangleModule DEFINITIONS ::= BEGIN

Rectangle ::= SEQUENCE {
    height  INTEGER,
    width   INTEGER
}

END
```

2. Compile it into the set of .c and .h files using asn1c compiler [ASN1C]:

```
asn1c -no-gen-example rectangle.asn
```

3. Alternatively, use the Online ASN.1 compiler [AONL] by uploading the **rectangle.asn** file into the Web form and unpacking the produced archive on your computer.

4. By this time, you should have gotten multiple files in the current directory, including the **Rectangle.c** and **Rectangle.h**.

5. Create a main() routine which takes the binary input file, decodes it as it were a BER-encoded Rectangle type, and prints out the text (XML) representation of the Rectangle type. Let's name the file **main.c**:

```
#include <stdio.h>
#include <sys/types.h>
#include <Rectangle.h>   /* Rectangle ASN.1 type */

int main(int ac, char **av) {
    char buf[1024];      /* Temporary buffer      */
    asn_dec_rval_t rval; /* Decoder return value  */
    Rectangle_t *rectangle = 0; /* Type to decode. Note this 0[1]! */
```

[1] Forgetting to properly initialize the pointer to a destination structure is a major source of support requests.

```
    FILE *fp;            /* Input file handler    */
    size_t size;         /* Number of bytes read  */
    char *filename;      /* Input file name */

    /* Require a single filename argument */
    if(ac != 2) {
        fprintf(stderr, "Usage: %s <file.ber>\n", av[0]);
        exit(1);
    } else {
        filename = av[1];
    }

    /* Open input file as read-only binary */
    fp = fopen(filename, "rb");
    if(!fp) {
        perror(filename);
        exit(1);
    }

    /* Read up to the buffer size */
    size = fread(buf, 1, sizeof(buf), fp);
    fclose(fp);
    if(!size) {
        fprintf(stderr, "%s: Empty or broken\n", filename);
        exit(1);
    }

    /* Decode the input buffer as Rectangle type */
    rval = ber_decode(0, &asn_DEF_Rectangle, (void **)&rectangle, buf, size);
    if(rval.code != RC_OK) {
        fprintf(stderr, "%s: Broken Rectangle encoding at byte %ld\n", filename,
            (long)rval.consumed);
        exit(1);
    }

    /* Print the decoded Rectangle type as XML */
    xer_fprint(stdout, &asn_DEF_Rectangle, rectangle);

    return 0; /* Decoding finished successfully */
}
```

6. Compile all files together using C compiler (varies by platform):

   `cc -I. -o` ***`rdecode`*** `*.c`

7. Done. You have just created the BER decoder of a Rectangle type, named **rdecode**!

1.4 Adding constraints to a "Rectangle"

This example shows how to add basic constraints to the ASN.1 specification and how to invoke the constraints validation code in your application.

1. Create a file named **rectangle.asn** with the following contents:

```
RectangleModuleWithConstraints DEFINITIONS ::= BEGIN

Rectangle ::= SEQUENCE {
    height  INTEGER (0..100), -- Value range constraint
    width   INTEGER (0..MAX)  -- Makes width non-negative
}

END
```

2. Compile the file according to procedures shown in section 1.3 on page 7.

3. Modify the Rectangle type processing routine (you can start with the main() routine shown in the section 1.3 on page 7) by placing the following snippet of code *before* encoding and/or *after* decoding the Rectangle type:

```
int ret;           /* Return value */
char errbuf[128];  /* Buffer for error message */
size_t errlen = sizeof(errbuf);  /* Size of the buffer */

/* ... here goes the Rectangle decoding code ... */

ret = asn_check_constraints(&asn_DEF_Rectangle, rectangle, errbuf, &errlen);
/* assert(errlen < sizeof(errbuf)); // you may rely on that */
if(ret) {
    fprintf(stderr, "Constraint validation failed: %s\n", errbuf);
    /* exit(...); // Replace with appropriate action */
 }

/* ... here goes the Rectangle encoding code ... */
```

4. Compile the resulting C code as shown in the previous chapters.

5. Test the constraints checking code by assigning integer value 101 to the **.height** member of the Rectangle structure, or a negative value to the **.width** member. The program will fail with "Constraint validation failed" message.

6. Done.

Chapter 2

ASN.1 Compiler

2.1 The asn1c compiler tool

The purpose of the ASN.1 compiler is to convert the specifications in ASN.1 notation into some other language, such as C.

The compiler reads the specification and emits a series of target language structures (C structs, unions, enums) describing the corresponding ASN.1 types. The compiler also creates the code which allows automatic serialization and deserialization of these structures using several standardized encoding rules (BER, DER, OER, PER, XER).

The compiler includes advanced features for handling complex specifications:

- **ENCODING-CONTROL directives** for customizing XER encoding (see section 6.2 on page 76)
- **Circular dependency detection and resolution** for complex type graphs (see section 6.3 on page 79)
- **Information Object Class support** for protocol specifications like F1AP (see section 6.4 on page 82)

Let's take the following ASN.1 example[2]:

```
RectangleModule DEFINITIONS ::= BEGIN

Rectangle ::= SEQUENCE {
    height  INTEGER,        -- Height of the rectangle
    width   INTEGER         -- Width of the rectangle
```

[2] Chapter 7 on page 89 provides a quick reference on the ASN.1 notation.

```
}

END
```

The asn1c compiler reads this ASN.1 definition and produce the following C type:

```
typedef struct Rectangle_s {
    long height;
    long width;
} Rectangle_t;
```

The asn1c compiler also creates the code for converting this structure into platform-independent wire representation and the decoder of such wire representation back into local, machine-specific type. These encoders and decoders are also called serializers and deserializers, marshallers and unmarshallers, or codecs.

Compiling ASN.1 modules into C codecs can be as simple as invoking `asn1c`: may be used to compile the ASN.1 modules:

`asn1c` *<modules.asn>*

If several ASN.1 modules contain interdependencies, all of the files must be specified altogether:

`asn1c` *<module1.asn> <module2.asn> ...*

The compiler **-E** and **-EF** options are used for testing the parser and the semantic fixer, respectively. These options will instruct the compiler to dump out the parsed (and fixed, if **-F** is involved) ASN.1 specification as it was understood by the compiler. It might be useful to check whether a particular syntactic construct is properly supported by the compiler.

`asn1c` **-EF** *<module-to-test.asn>*

The **-P** option is used to dump the compiled output on the screen instead of creating a bunch of .c and .h files on disk in the current directory. You would probably want to start with **-P** option instead of creating a mess in your current directory. Another option, **-R**, asks compiler to only generate the files which need to be generated, and suppress linking in the numerous support files.

Print the compiled output instead of creating multiple source files:

`asn1c` **-P** *<module-to-compile-and-print.asn>*

2.2 Compiler output

The `asn1c` compiler produces a number of files:

- A set of .c and .h files for each type defined in the ASN.1 specification. These files will be named similarly to the ASN.1 types (**Rectangle.c** and **Rectangle.h** for the Rectangle-Module ASN.1 module defined in the beginning of this document).
- A set of helper .c and .h files which contain the generic encoders, decoders and other useful routines. Sometimes they are referred to by the term *skeletons*. There will be quite a few of them, some of them are not even always necessary, but the overall amount of code after compilation will be rather small anyway.
- A **Makefile.am.libasncodecs** file which explicitly lists all the generated files. This makefile can be used on its own to build the just the codec library.
- A **converter-example.c** file containing the *int main()* function with a fully functioning encoder and data format converter. It can convert a given PDU between BER, XER, OER and PER. At some point you will want to replace this file with your own file containing the *int main()* function.
- A **converter-example.mk** file which binds together **Makefile.am.libasncodecs** and **converter-example.c** to build a versatile converter and debugger for your data formats.

It is possible to compile everything with just a couple of instructions:

```
asn1c -pdu=Rectangle *.asn
make -f converter-example.mk                    # If you use
   `make`
```

or

```
asn1c *.asn
cc -I. -DPDU=Rectangle -o rectangle.exe *.c   # ... or like this
```

Refer to the chapter 1 on page 4 for a sample *int main()* function if you want some custom logic and not satisfied with the supplied *converter-example.c*.

2.3 Command line options

The following table summarizes the `asn1c` command line options.

Stage Selection Options	Description
`-E`	Stop after the parsing stage and print the reconstructed ASN.1 specification code to the standard output.
`-F`	Used together with -E, instructs the compiler to stop after the ASN.1 syntax tree fixing stage and dump the reconstructed ASN.1 specification to the standard output.
`-P`	Dump the compiled output to the standard output instead of creating the target language files on disk.
`--rust`	Generate a standalone Cargo project using safe Rust and a copied runtime. The current subset provides UPER encode/decode and CER encoding for top-level BOOLEAN, NULL, and finite nonextensible INTEGER ranges fitting `i64`. Build the generated project with Cargo; Rust does not currently decode CER.
`-R`	Restrict the compiler to generate only the ASN.1 tables, omitting the usual support code.
`-S <directory>`	Use the specified directory with ASN.1 skeleton files.
`-X`	Generate the XML DTD for the specified ASN.1 modules.

Warning Options	Description
`-Werror`	Treat warnings as errors; abort if any warning is produced.
`-Wdebug-parser`	Enable the parser debugging during the ASN.1 parsing stage.
`-Wdebug-lexer`	Enable the lexer debugging during the ASN.1 parsing stage.
`-Wdebug-fixer`	Enable the ASN.1 syntax tree fixer debugging during the fixing stage.
`-Wdebug-compiler`	Enable debugging during the actual compile time.

Language Options	**Description**
`-fbless-SIZE`	Allow SIZE() constraint for INTEGER, ENUMERATED, and other types for which this constraint is normally prohibited by the standard. This is a violation of an ASN.1 standard and compiler may fail to produce the meaningful code.
`-fcompound-names`	Use complex names for C structures. Using complex names prevents name clashes in case the module reuses the same identifiers in multiple contexts.
`-findirect-choice`	When generating code for a CHOICE type, compile the CHOICE members as indirect pointers instead of declaring them inline. Also affects Information Object Class OPEN_TYPE pointer generation. Consider using this option together with `-fno-include-deps` to prevent circular references.
`-fincludes-quoted`	Generate #include lines in "double" instead of <angle> quotes.
`-fknown-extern-type=<`*name*`>`	Pretend the specified type is known. The compiler will assume the target language source files for the given type have been provided manually.
`-fline-refs`	Include ASN.1 module's line numbers in generated code comments.
`-fno-constraints`	Do not generate the ASN.1 subtype constraint checking code. This may produce a shorter executable.
`-fno-include-deps`	Do not generate the courtesy #include lines for non-critical dependencies.

`-fprefix=<prefix>`	Add the specified prefix to all generated type names and filenames. This helps avoid naming conflicts in several scenarios: (1) **System header conflicts**: On case-insensitive filesystems (macOS HFS+, Windows), ASN.1 types like `Time` may collide with system header `<time.h>`. asn1c now automatically disambiguates these generated filenames (for example, `Time.h` becomes `asn1c_time.h` when no explicit prefix is set). Using `-fprefix=ASN1_` still generates `ASN1_Time.h` when a project-specific namespace is desired. (2) **Multiple ASN.1 modules**: When generating code for multiple ASN.1 syntaxes with type name clashes, a prefix prevents symbol collisions. (3) **Integration with existing code**: Prefixes help avoid conflicts with existing types in your codebase. When used with `-pdu` options, the prefix is automatically applied to PDU definitions in `converter-example.mk`. For example, `-fprefix=ASN1_ -pdu=Certificate` generates `-DPDU=ASN1_Certificate`. With `-pdu=all` or `-pdu=auto`, the generated `converter-example.mk` includes instructions for selecting specific prefixed PDU types.
`-funnamed-unions`	Enable unnamed unions in the definitions of target language's structures.
`-fwide-types`	Use the unbounded data types (INTEGER_t, ENUMERATED_t, REAL_t) by default, instead of machine's native data types (long, double).
`-flong-size=<bits>`	Select the target C `long` model used by the default native INTEGER storage policy. Values are 32 and 64; the default is the historical portable 32-bit model. Use this option when generated code is produced on one platform for a target with a different `long` size.

`-finteger-native-type=<mode>`	Select the fixed-width native C storage policy for constrained ASN.1 INTEGER types. Modes are `auto`, `int32`, `uint32`, `int64`, and `uint64`; the default is `auto`. The `auto` mode preserves the traditional `long`/`unsigned long`/INTEGER_t selection. The explicit modes permit `int32_t`, `uint32_t`, `int64_t`, or `uint64_t` storage only when the INTEGER constraint fits that type. Larger or incompatible ranges are generated as INTEGER_t, while their constraint bounds are still preserved, including unsigned 64-bit and arbitrary-precision bounds.

Codecs Generation Options	**Description**
`-gen-CER`	Generate CER encoding support and its required BER-family dependencies. CER support is enabled by default.
`-no-gen-CER`	Do not generate CER encoding support. The BER decoder may still accept CER-compatible input.
`-no-gen-BER`	Do not generate BER-family support, including dependencies used by DER and CER encoding.
`-no-gen-XER`	Do not generate the XML Encoding Rules (XER, X.693) support code.
`-no-gen-OER`	Do not generate the Octet Encoding Rules (OER, X.696) support code.
`-no-gen-CBOR`	Do not generate the Concise Binary Object Representation (CBOR, RFC 8949) support code. By default, CBOR support code is generated.
`-no-gen-UPER`	Do not generate the Unaligned Packed Encoding Rules (PER, X.691) support code.
`-no-gen-APER`	Do not generate the Aligned Packed Encoding Rules (PER, X.691) support code.
`-no-gen-print`	Do not generate the print code.
`-no-gen-random-fill`	Do not generate the random fill code.

`–no–gen–example`	Do not generate the ASN.1 format converter example.
`–pdu={`**`all`**`|`**`auto`**`|`*Type*`}`	Create a PDU table for specified types, or discover the Protocol Data Units automatically. In case of `–pdu=`**`all`**, all ASN.1 types defined in all modules will form a PDU table. In case of `–pdu=`**`auto`**, all types not referenced by any other type will form a PDU table. If *Type* is an ASN.1 type identifier, it is added to a PDU table. The last form may be specified multiple times.
`–fgen–only–pdu–deps`	Generate code only for types that are dependencies of `–pdu` types. This includes types referenced in Information Object Class tables, parameterized types, and constraint-based IOC references. Useful for large specifications like F1AP where only a subset of types is needed. See section 6.4 on page 82.

Output Options	**Description**
`–print–class–matrix`	When –EF options are given, this option instructs the compiler to print out the collected Information Object Class matrix.
`–print–constraints`	With –EF, this option instructs the compiler to explain its internal understanding of subtype constraints.
`–print–lines`	Generate "`–– #line`" comments in –E output.

Chapter 3

API reference

The functions described in this chapter are to be used by the application programmer. These functions won't likely change change or get removed until the next major release.

The API calls not listed here are not public and should not be used by the application level code.

3.1 ASN_STRUCT_FREE() macro

Synopsis

```
#define ASN_STRUCT_FREE(type_descriptor, struct_ptr)
```

Description

Recursively releases memory occupied by the structure described by the **`type_descriptor`** and referred to by the **`struct_ptr`** pointer.

Does nothing when **`struct_ptr`** is NULL.

Return values

Does not return a value.

Example

```
Rectangle_t *rect = ...;
ASN_STRUCT_FREE(asn_DEF_Rectangle, rect);
```

3.2 ASN_STRUCT_RESET() macro

Synopsis

```
#define ASN_STRUCT_RESET(type_descriptor, struct_ptr)
```

Description

Recursively releases memory occupied by the members of the structure described by the **`type_descriptor`** and referred to by the **`struct_ptr`** pointer.

Does not release the memory pointed to by **`struct_ptr`** itself. Instead it clears the memory block by filling it out with 0 bytes.

Does nothing when **`struct_ptr`** is NULL.

Return values

Does not return a value.

Example

```
struct my_figure {       /* The custom structure */
    int flags;           /* <some custom member> */
    /* The type is generated by the ASN.1 compiler */
    Rectangle_t rect;
    /* other members of the structure */
};

struct my_figure *fig = ...;
ASN_STRUCT_RESET(asn_DEF_Rectangle, &fig->rect);
```

3.3 asn_check_constraints()

Synopsis

```
int asn_check_constraints(
    const asn_TYPE_descriptor_t *type_descriptor,
    const void *struct_ptr, /* Target language's structure */
    char *errbuf,           /* Returned error description */
    size_t *errlen          /* Length of the error description */
);
int asn_check_constraints_with_syntax(
    const asn_TYPE_descriptor_t *type_descriptor,
    const void *struct_ptr,
    int syntax, char *errbuf, size_t *errlen
);
```

Description

Validate a given structure according to the ASN.1 constraints. If **errbuf** and **errlen** are given, they shall point to the appropriate buffer space and its length before calling this function. Alternatively, they could be passed as **NULL**s. If constraints validation fails, **errlen** will contain the actual number of bytes used in **errbuf** to encode an error message. The message is going to be properly 0-terminated. For **CONTAINING** constraints, **asn_check_constraints()** uses BER. Use **asn_check_constraints_with_syntax()** when the contained value uses the same encoding rules as another transfer syntax.

Return values

This function returns 0 in case all ASN.1 constraints are met and -1 if one or more ASN.1 constraints were violated.

Example

```
Rectangle_t *rect = ...;

char errbuf[128];  /* Buffer for error message */
size_t errlen = sizeof(errbuf);  /* Size of the buffer */

int ret = asn_check_constraints(&asn_DEF_Rectangle, rectangle, errbuf, &errlen);
/* assert(errlen < sizeof(errbuf)); // Guaranteed: you may rely on that */
```

```
if(ret) {
    fprintf(stderr, "Constraint validation failed: %s\n", errbuf);
}
```

3.4 asn_decode()

Synopsis

```
asn_dec_rval_t asn_decode(
    const asn_codec_ctx_t *opt_codec_ctx,
    enum asn_transfer_syntax syntax,
    const asn_TYPE_descriptor_t *type_descriptor,
    void **struct_ptr_ptr,/* Pointer to a target structure's ptr */
    const void *buffer,   /* Data to be decoded */
    size_t size           /* Size of that buffer */
);
```

Description

The **asn_decode()** function parses the data given by the **buffer** and **size** arguments. The encoding rules are specified in the **syntax** argument and the type to be decoded is specified by the **type_descriptor**.

The **struct_ptr_ptr** must point to the memory location which contains the pointer to the structure being decoded. Initially the ***struct_ptr_ptr** pointer is typically set to 0. In that case, **asn_decode()** will dynamically allocate memory for the structure and its members as needed during the parsing. If ***struct_ptr_ptr** already points to some memory, the **asn_decode()** will allocate the subsequent members as needed during the parsing.

Return values

Upon unsuccessful termination, the ***struct_ptr_ptr** may contain partially decoded data. This data may be useful for debugging (such as by using **asn_fprint()**). Don't forget to discard the unused partially decoded data by calling **ASN_STRUCT_FREE()** or **ASN_STRUCT_RESET()**.

The function returns a compound structure:

```
typedef struct {
    enum {
        RC_OK,          /* Decoded successfully */
        RC_WMORE,       /* More data expected, call again */
        RC_FAIL         /* Failure to decode data */
    } code;             /* Result code */
    size_t consumed;    /* Number of bytes consumed */
} asn_dec_rval_t;
```

The **.code** member specifies the decoding outcome.

RC_OK	Decoded successfully and completely
RC_WMORE	More data expected, call again
RC_FAIL	Failed for good

The **.consumed** member specifies the amount of **buffer** data that was used during parsing, irrespectively of the **.code**.

The **.consumed** value is in bytes, even for PER decoding. For PER, use **uper_decode()** in case you need to get the number of consumed bits.

Restartability

Some transfer syntax parsers (such as ATS_BER) support restartability.

That means that in case the buffer has less data than expected, the **asn_decode()** will process whatever is available and ask for more data to be provided using the RC_WMORE return **.code**.

Note that in the RC_WMORE case the decoder may have processed less data than it is available in the buffer, which means that you must be able to arrange the next buffer to contain the unprocessed part of the previous buffer.

The **RC_WMORE** code may still be returned by parser not supporting restartabilty. In such cases, the partially decoded structure shall be discarded and the next invocation should use the extended buffer to parse from the very beginning.

Example

```
Rectangle_t *rect = 0;    /* Note this 0[1]! */
asn_dec_rval_t rval;
rval = asn_decode(0, ATS_BER, &asn_DEF_Rectangle, (void **)&rect, buffer, buf_size);
switch(rval.code) {
case RC_OK:
    asn_fprint(stdout, &asn_DEF_Rectangle, rect);
    ASN_STRUCT_FREE(&asn_DEF_Rectangle, rect);
    break;
case RC_WMORE:
case RC_FAIL:
default:
    ASN_STRUCT_FREE(&asn_DEF_Rectangle, rect);
    break;
}
```

[1] Forgetting to properly initialize the pointer to a destination structure is a major source of support requests.

See also

asn_fprint() at page 32.

3.5 asn_encode()

Synopsis

```
#include <asn_application.h>

asn_enc_rval_t asn_encode(
    const asn_codec_ctx_t *opt_codec_ctx,
    enum asn_transfer_syntax syntax,
    const asn_TYPE_descriptor_t *type_to_encode,
    const void *structure_to_encode,
    asn_app_consume_bytes_f *callback, void *callback_key);
```

Description

The **`asn_encode()`** function serializes the given **`structure_to_encode`** using the chosen ASN.1 transfer **`syntax`**.

During serialization, a user-specified **`callback`** is invoked zero or more times with bytes of data to add to the output stream (if any), and the **`callback_key`**. The signature for the callback is as follows:

```
typedef int(asn_app_consume_bytes_f)(const void *buffer, size_t
   size, void *callback_key);
```

Return values

The function returns a compound structure:

```
typedef struct {
    ssize_t encoded;
    const asn_TYPE_descriptor_t *failed_type;
    const void *structure_ptr;
} asn_enc_rval_t;
```

In case of unsuccessful encoding, the **`.encoded`** member is set to -1 and the other members of the compound structure point to where the encoding has failed to proceed further.

In case encoding is successful, the **`.encoded`** member specifies the size of the serialized output.

The serialized output size is returned in **bytes** irrespectively of the ASN.1 transfer **syntax** chosen.[1]

On error (when **.encoded** is set to -1), the **errno** is set to one of the following values:

EINVAL	Incorrect parameters to the function, such as NULLs
ENOENT	Encoding transfer syntax is not defined (for this type)
EBADF	The structure has invalid form or content constraint failed
EIO	The callback has returned negative value during encoding

Example

```
static int
save_to_file(const void *data, size_t size, void *key) {
    FILE *fp = key;
    return (fwrite(data, 1, size, fp) == size) ? 0 : -1;
}

Rectangle_t *rect = ...;
FILE *fp = ...;
asn_enc_rval_t er;
er = asn_encode(0, ATS_DER, &asn_DEF_Rectangle, rect, save_to_file, fp);
if(er.encoded == -1) {
   fprintf(stderr, "Failed to encode %s\n", asn_DEF_Rectangle.name);
} else {
   fprintf(stderr, "%s encoded in %zd bytes\n", asn_DEF_Rectangle.name, er.encoded);
}
```

[1] This is different from some lower level encoding functions, such as **uper_encode()**, which returns the number of encoded *bits* instead of bytes.

3.6 asn_encode_to_buffer()

Synopsis

```
#include <asn_application.h>

asn_enc_rval_t asn_encode_to_buffer(
    const asn_codec_ctx_t *opt_codec_ctx,
    enum asn_transfer_syntax syntax,
    const asn_TYPE_descriptor_t *type_to_encode,
    const void *structure_to_encode,
    void *buffer, size_t buffer_size);
```

Description

The **asn_encode_to_buffer()** function serializes the given **structure_to_encode** using the chosen ASN.1 transfer **syntax**.

The function places the serialized data into the given **buffer** of size **buffer_size**.

Return values

The function returns a compound structure:

```
typedef struct {
    ssize_t encoded;
    const asn_TYPE_descriptor_t *failed_type;
    const void *structure_ptr;
} asn_enc_rval_t;
```

In case of unsuccessful encoding, the **.encoded** member is set to -1 and the other members of the compound structure point to where the encoding has failed to proceed further.

In case encoding is successful, the **.encoded** member specifies the size of the serialized output.

The serialized output size is returned in **bytes** irrespectively of the ASN.1 transfer **syntax** chosen.[1]

[1] This is different from some lower level encoding functions, such as **uper_encode()**, which returns the number of encoded *bits* instead of bytes.

If **.encoded** size exceeds the specified **buffer_size**, the serialization effectively failed due to insufficient space. The function will succeed if subsequently called with buffer size no less than the returned **.encoded** size. This behavior modeled after **snprintf()**.

On error (when **.encoded** is set to -1), the **errno** is set to one of the following values:

EINVAL	Incorrect parameters to the function, such as NULLs
ENOENT	Encoding transfer syntax is not defined (for this type)
EBADF	The structure has invalid form or content constraint failed

Example

```
Rectangle_t *rect = ...;
uint8_t buffer[128];
asn_enc_rval_t er;
er = asn_encode_to_buffer(0, ATS_DER, &asn_DEF_Rectangle, rect, buffer, sizeof(buffer));
if(er.encoded == -1) {
   fprintf(stderr, "Serialization of %s failed.\n", asn_DEF_Rectangle.name);
} else if(er.encoded > sizeof(buffer)) {
   fprintf(stderr, "Buffer of size %zu is too small for %s, need %zu\n",
       buf_size, asn_DEF_Rectangle.name, er.encoded);
}
```

See also

asn_encode_to_new_buffer() at page 30.

3.7 asn_encode_to_new_buffer()

Synopsis

```
#include <asn_application.h>

typedef struct {
    void *buffer;   /* NULL if failed to encode. */
    asn_enc_rval_t result;
} asn_encode_to_new_buffer_result_t;
asn_encode_to_new_buffer_result_t asn_encode_to_new_buffer(
    const asn_codec_ctx_t *opt_codec_ctx,
    enum asn_transfer_syntax syntax,
    const asn_TYPE_descriptor_t *type_to_encode,
    const void *structure_to_encode);
```

Description

The **asn_encode_to_new_buffer()** function serializes the given **structure_to_encode** using the chosen ASN.1 transfer **syntax**.

The function places the serialized data into the newly allocated buffer which it returns in a compound structure.

Return values

On failure, the **.buffer** is set to **NULL** and **.result.encoded** is set to -1. The global **errno** is set to one of the following values:

EINVAL	Incorrect parameters to the function, such as NULLs
ENOENT	Encoding transfer syntax is not defined (for this type)
EBADF	The structure has invalid form or content constraint failed
ENOMEM	Memory allocation failed due to system or internal limits

On success, the **.result.encoded** is set to the number of **bytes** that it took to serialize the structure. The **.buffer** contains the serialized content. The user is responsible for freeing the **.buffer**.

Example

```
asn_encode_to_new_buffer_result_t res;
res = asn_encode_to_new_buffer(0, ATS_DER, &asn_DEF_Rectangle, rect);
if(res.buffer) {
    /* Encoded successfully. */
    free(res.buffer);
} else {
    fprintf(stderr, "Failed to encode %s, estimated %zd bytes\n",
        asn_DEF_Rectangle.name, res.result.encoded);
}
```

See also

asn_encode_to_buffer() at page 28.

3.8 asn_fprint()

Synopsis

```
int asn_fprint(FILE *stream,    /* Destination file */
    const asn_TYPE_descriptor_t *type_descriptor,
    const void *struct_ptr      /* Structure to be printed */
);
```

Description

The **asn_fprint()** function prints human readable description of the target language's structure into the file stream specified by **stream** pointer.

The output format does not conform to any standard.

The **asn_fprint()** function attempts to produce a valid output even for incomplete and broken structures, which makes it more suitable for debugging complex cases than **xer_fprint()**.

Return values

0	Output was successfully made
-1	Error printing out the structure

Example

```
Rectangle_t *rect = ...;
asn_fprint(stdout, &asn_DEF_Rectangle, rect);
```

See also

xer_fprint() at page 57.

3.9 asn_random_fill()

Synopsis

```
int asn_random_fill(
    const asn_TYPE_descriptor_t *type_descriptor,
    void **struct_ptr_ptr,/* Pointer to a target structure's ptr */
    size_t approx_max_length_limit
);
int asn_random_fill_with_syntax(
    const asn_TYPE_descriptor_t *type_descriptor,
    void **struct_ptr_ptr,
    size_t approx_max_length_limit, int syntax
);
```

Description

Create or initialize a structure with random contents, according to the type specification and optional member constraints. The **asn_random_fill_with_syntax()** variant encodes values of types used by **CONTAINING** constraints using the specified transfer syntax.

For best results the code should be generated without –no–gen–UPER options to asn1c. Making PER constraints code available in runtime will make **asn_random_fill** explore the edges of PER-visible constraints and sometimes break out of extensible contstraints' ranges.

The **asn_random_fill()** function has a bias to generate edge case values. This property makes it useful for debugging the application level code and for security testing, as random data can be a good seed to fuzzing.

The **approx_max_length_limit** specifies the approximate limit of the resulting structure in units closely resembling bytes. The actual result might be several times larger or smaller than the given length limit. A rule of thumb way to select the initial value for this parameter is to take a typical structure and use twice its DER output size.

Return values

0	Structure was properly initialized with random data
-1	Failure to initialize the structure with random data

3.10 ber_decode()

Synopsis

```
asn_dec_rval_t ber_decode(
    const asn_codec_ctx_t *opt_codec_ctx,
    const asn_TYPE_descriptor_t *type_descriptor,
    void **struct_ptr_ptr,/* Pointer to a target structure's ptr */
    const void *buffer,   /* Data to be decoded */
    size_t size           /* Size of that buffer */
);
```

Description

Decode BER, DER and CER data (Basic Encoding Rules, Distinguished Encoding Rules, Canonical Encoding Rules), as defined by ITU-T X.690.

DER and CER are different subsets of BER.

*Consider using a more generic function **`asn_decode(ATS_BER)`**.*

Return values

Upon unsuccessful termination, the **`*struct_ptr_ptr`** may contain partially decoded data. This data may be useful for debugging (such as by using **`asn_fprint()`**). Don't forget to discard the unused partially decoded data by calling **`ASN_STRUCT_FREE()`** or **`ASN_STRUCT_RESET()`**.

The function returns a compound structure:

```
typedef struct {
    enum {
        RC_OK,          /* Decoded successfully */
        RC_WMORE,       /* More data expected, call again */
        RC_FAIL         /* Failure to decode data */
    } code;             /* Result code */
    size_t consumed;    /* Number of bytes consumed */
} asn_dec_rval_t;
```

The **`.code`** member specifies the decoding outcome.

RC_OK	Decoded successfully and completely
RC_WMORE	More data expected, call again
RC_FAIL	Failed for good

The **.consumed** member specifies the amount of **buffer** data that was used during parsing, irrespectively of the **.code**.

The **.consumed** value is in bytes. CER input is processed by the BER decoder; this call does not validate CER's additional canonical restrictions.

Restartability

The **ber_decode()** function is restartable (stream-oriented). That means that in case the buffer has less data than expected, the decoder will process whatever is available and ask for more data to be provided using the RC_WMORE return **.code**.

Note that in the RC_WMORE case the decoder may have processed less data than it is available in the buffer, which means that you must be able to arrange the next buffer to contain the unprocessed part of the previous buffer.

See also

der_encode() at page 36.

3.11 der_encode

Synopsis

```
asn_enc_rval_t der_encode(
    const asn_TYPE_descriptor_t *type_descriptor,
    const void *structure_to_encode,
    asn_app_consume_bytes_f *callback,
    void *callback_key
```

Description

The **der_encode()** function serializes the given **structure_to_encode** using the DER transfer syntax (a variant of BER, Basic Encoding Rules).

During serialization, a user-specified **callback** is invoked zero or more times with bytes of data to add to the output stream (if any), and the **callback_key**. The signature for the callback is as follows:

```
typedef int(asn_app_consume_bytes_f)(const void *buffer, size_t
   size, void *callback_key);
```

*Consider using a more generic function **asn_encode(ATS_DER)**.*

Return values

The function returns a compound structure:

```
typedef struct {
    ssize_t encoded;
    const asn_TYPE_descriptor_t *failed_type;
    const void *structure_ptr;
} asn_enc_rval_t;
```

In case of unsuccessful encoding, the **.encoded** member is set to -1 and the other members of the compound structure point to where the encoding has failed to proceed further.

In case encoding is successful, the **.encoded** member specifies the size of the serialized output.

The serialized output size is returned in **bytes**.

Example

```
static int
save_to_file(const void *data, size_t size, void *key) {
    FILE *fp = key;
    return (fwrite(data, 1, size, fp) == size) ? 0 : -1;
}

Rectangle_t *rect = ...;
FILE *fp = ...;
asn_enc_rval_t er;
er = der_encode(&asn_DEF_Rectangle, rect, save_to_file, fp);
if(er.encoded == -1) {
   fprintf(stderr, "Failed to encode %s\n", asn_DEF_Rectangle.name);
} else {
   fprintf(stderr, "%s encoded in %zd bytes\n", asn_DEF_Rectangle.name, er.encoded);
}
```

See also

ber_decode() at page 34, **asn_decode(ATS_BER)** at page 23.

3.12 der_encode_to_buffer()

Synopsis

```
asn_enc_rval_t der_encode_to_buffer(
    const asn_TYPE_descriptor_t *type_descriptor,
    const void *structure_to_encode,
    void *buffer, size_t buffer_size);
```

Description

The **der_encode_to_buffer()** function serializes the given **structure_to_encode** using the DER transfer syntax (a variant of BER, Basic Encoding Rules).

The function places the serialized data into the given **buffer** of size **buffer_size**.

*Consider using a more generic function **asn_encode_to_buffer(ATS_DER)**.*

Return values

The function returns a compound structure:

```
typedef struct {
    ssize_t encoded;
    const asn_TYPE_descriptor_t *failed_type;
    const void *structure_ptr;
} asn_enc_rval_t;
```

In case of unsuccessful encoding, the **.encoded** member is set to -1 and the other members of the compound structure point to where the encoding has failed to proceed further.

In case encoding is successful, the **.encoded** member specifies the size of the serialized output.

The serialized output size is returned in **bytes**.

The **.encoded** never exceeds the available buffer size.[1] If the **buffer_size** is not sufficient, the **.encoded** will be set to -1 and encoding would fail.

Example

[1] This behavior is different from **asn_encode_to_buffer()**.

```
Rectangle_t *rect = ...;
uint8_t buffer[128];
asn_enc_rval_t er;
er = der_encode_to_buffer(&asn_DEF_Rectangle, rect, buffer, sizeof(buffer));
if(er.encoded == -1) {
   fprintf(stderr, "Serialization of %s failed.\n", asn_DEF_Rectangle.name);
}
```

See also

ber_decode() at page 34, **asn_decode(ATS_BER)** at page 23, **asn_encode_to_buffer(ATS_DER)** at page 28.

3.13 oer_decode()

Synopsis

```
asn_dec_rval_t oer_decode(
    const asn_codec_ctx_t *opt_codec_ctx,
    const asn_TYPE_descriptor_t *type_descriptor,
    void **struct_ptr_ptr,/* Pointer to a target structure's ptr */
    const void *buffer,   /* Data to be decoded */
    size_t size           /* Size of that buffer */
);
```

Description

Decode the BASIC-OER and CANONICAL-OER (Octet Encoding Rules), as defined by ITU-T X.696.

*Consider using a more generic function **`asn_decode(ATS_BASIC_OER)`**.*

Return values

Upon unsuccessful termination, the **`*struct_ptr_ptr`** may contain partially decoded data. This data may be useful for debugging (such as by using **`asn_fprint()`**). Don't forget to discard the unused partially decoded data by calling **`ASN_STRUCT_FREE()`** or **`ASN_STRUCT_RESET()`**.

The function returns a compound structure:

```
typedef struct {
    enum {
        RC_OK,          /* Decoded successfully */
        RC_WMORE,       /* More data expected, call again */
        RC_FAIL         /* Failure to decode data */
    } code;             /* Result code */
    size_t consumed;    /* Number of bytes consumed */
} asn_dec_rval_t;
```

The **`.code`** member specifies the decoding outcome.

RC_OK	Decoded successfully and completely
RC_WMORE	More data expected, call again
RC_FAIL	Failed for good

The **`.consumed`** member specifies the amount of **`buffer`** data that was used during parsing, irrespectively of the **`.code`**.

The **`.consumed`** value is in bytes.

Restartability

The **`oer_decode()`** function is restartable (stream-oriented). That means that in case the buffer has less data than expected, the decoder will process whatever is available and ask for more data to be provided using the RC_WMORE return **`.code`**.

Note that in the RC_WMORE case the decoder may have processed less data than it is available in the buffer, which means that you must be able to arrange the next buffer to contain the unprocessed part of the previous buffer.

3.14 oer_encode()

Synopsis

```
asn_enc_rval_t oer_encode(
    const asn_TYPE_descriptor_t *type_descriptor,
    const void *structure_to_encode,
    asn_app_consume_bytes_f *callback,
    void *callback_key);
```

Description

The **oer_encode()** function serializes the given **structure_to_encode** using the CANONICAL-OER transfer syntax (Octet Encoding Rules, ITU-T X.691).

During serialization, a user-specified **callback** is invoked zero or more times with bytes of data to add to the output stream (if any), and the **callback_key**. The signature for the callback is as follows:

```
typedef int(asn_app_consume_bytes_f)(const void *buffer, size_t
   size, void *callback_key);
```

*Consider using a more generic function **asn_encode(ATS_CANONICAL_OER)**.*

Return values

The function returns a compound structure:

```
typedef struct {
    ssize_t encoded;
    const asn_TYPE_descriptor_t *failed_type;
    const void *structure_ptr;
} asn_enc_rval_t;
```

In case of unsuccessful encoding, the **.encoded** member is set to -1 and the other members of the compound structure point to where the encoding has failed to proceed further.

In case encoding is successful, the **.encoded** member specifies the size of the serialized output.

The serialized output size is returned in **bytes**.

Example

```
static int
save_to_file(const void *data, size_t size, void *key) {
    FILE *fp = key;
    return (fwrite(data, 1, size, fp) == size) ? 0 : -1;
}

Rectangle_t *rect = ...;
FILE *fp = ...;
asn_enc_rval_t er;
er = oer_encode(&asn_DEF_Rectangle, rect, save_to_file, fp);
if(er.encoded == -1) {
   fprintf(stderr, "Failed to encode %s\n", asn_DEF_Rectangle.name);
} else {
   fprintf(stderr, "%s encoded in %zd bytes\n", asn_DEF_Rectangle.name, er.encoded);
}
```

See also

asn_encode(ATS_CANONICAL_OER) at page 26.

3.15 oer_encode_to_buffer()

Synopsis

```
asn_enc_rval_t oer_encode_to_buffer(
    const asn_TYPE_descriptor_t *type_descriptor,
    const asn_oer_constraints_t *constraints,
    const void *structure_to_encode,
    void *buffer, size_t buffer_size);
```

Description

The **oer_encode_to_buffer()** function serializes the given **structure_to_encode** using the CANONICAL-OER transfer syntax (Octet Encoding Rules, ITU-T X.691).

The function places the serialized data into the given **buffer** of size **buffer_size**.

*Consider using a more generic function **asn_encode_to_buffer(ATS_CANONICAL_OER)**.*

Return values

The function returns a compound structure:

```
typedef struct {
    ssize_t encoded;
    const asn_TYPE_descriptor_t *failed_type;
    const void *structure_ptr;
} asn_enc_rval_t;
```

In case of unsuccessful encoding, the **.encoded** member is set to -1 and the other members of the compound structure point to where the encoding has failed to proceed further.

In case encoding is successful, the **.encoded** member specifies the size of the serialized output.

The serialized output size is returned in **bytes**.

The **.encoded** never exceeds the available buffer size.[1] If the **buffer_size** is not sufficient, the **.encoded** will be set to -1 and encoding would fail.

[1] This behavior is different from **asn_encode_to_buffer()**.

Example

```
Rectangle_t *rect = ...;
uint8_t buffer[128];
asn_enc_rval_t er;
er = oer_encode_to_buffer(&asn_DEF_Rectangle, 0, rect, buffer, sizeof(buffer));
if(er.encoded == -1) {
   fprintf(stderr, "Serialization of %s failed.\n", asn_DEF_Rectangle.name);
}
```

See also

ber_decode() at page 34, **asn_decode(ATS_BER)** at page 23, **asn_encode_to_buffer(ATS_DER)** at page 28.

3.16 uper_decode()

Synopsis

```
asn_dec_rval_t uper_decode(
    const asn_codec_ctx_t *opt_codec_ctx,
    const asn_TYPE_descriptor_t *type_descriptor,
    void **struct_ptr_ptr,/* Pointer to a target structure's ptr */
    const void *buffer,   /* Data to be decoded */
    size_t size,          /* Size of the input data buffer, bytes */
    int skip_bits,        /* Number of unused leading bits, 0..7 */
    int unused_bits       /* Number of unused tailing bits, 0..7 */
);
```

Description

Decode the Unaligned BASIC or CANONICAL PER (Packed Encoding Rules), as defined by ITU-T X.691.

*Consider using a more generic function **`asn_decode(ATS_UNALIGNED_BASIC_PER)`**.*

Return values

Upon unsuccessful termination, the **`*struct_ptr_ptr`** may contain partially decoded data. This data may be useful for debugging (such as by using **`asn_fprint()`**). Don't forget to discard the unused partially decoded data by calling **`ASN_STRUCT_FREE()`** or **`ASN_STRUCT_RESET()`**.

The function returns a compound structure:

```
typedef struct {
    enum {
        RC_OK,          /* Decoded successfully */
        RC_WMORE,       /* More data expected, call again */
        RC_FAIL         /* Failure to decode data */
    } code;             /* Result code */
    size_t consumed;    /* Number of bytes consumed */
} asn_dec_rval_t;
```

The **`.code`** member specifies the decoding outcome.

RC_OK	Decoded successfully and completely
RC_WMORE	More data expected, call again
RC_FAIL	Failed for good

The **.consumed** member specifies the amount of **buffer** data that was used during parsing, irrespectively of the **.code**.

Note that the **.consumed** value is in bits. Use **(.consumed+7)/8** to convert to bytes.

Restartability

The **uper_decode()** function is not restartable. Failures are final.

3.17 uper_decode_complete()

Synopsis

```
asn_dec_rval_t uper_decode_complete(
    const asn_codec_ctx_t *opt_codec_ctx,
    const asn_TYPE_descriptor_t *type_descriptor,
    void **struct_ptr_ptr,/* Pointer to a target structure's ptr */
    const void *buffer,   /* Data to be decoded */
    size_t size           /* Size of data buffer */
);
```

Description

Decode a "Production of a complete encoding", according to ITU-T X.691 (08/2015) #11.1.

*Consider using a more generic function **`asn_decode(ATS_UNALIGNED_BASIC_PER)`**.*

Return values

Upon unsuccessful termination, the **`*struct_ptr_ptr`** may contain partially decoded data. This data may be useful for debugging (such as by using **`asn_fprint()`**). Don't forget to discard the unused partially decoded data by calling **`ASN_STRUCT_FREE()`** or **`ASN_STRUCT_RESET()`**.

The function returns a compound structure:

```
typedef struct {
    enum {
        RC_OK,          /* Decoded successfully */
        RC_WMORE,       /* More data expected, call again */
        RC_FAIL         /* Failure to decode data */
    } code;             /* Result code */
    size_t consumed;    /* Number of bytes consumed */
} asn_dec_rval_t;
```

The **`.code`** member specifies the decoding outcome.

RC_OK	Decoded successfully and completely
RC_WMORE	More data expected, call again
RC_FAIL	Failed for good

The **`.consumed`** member specifies the amount of **`buffer`** data that was used during parsing, irrespectively of the **`.code`**.

The the **`.consumed`** value is returned in whole *bytes* (NB).

Restartability

The **`uper_decode_complete()`** function is not restartable. Failures are final.

The complete encoding contains at least one byte, so on success **`.consumed`** will be greater or equal to 1.

3.18 uper_encode()

3.19 uper_encode_to_buffer()

3.20 uper_encode_to_new_buffer()

3.21 xer_decode()

Synopsis

```
asn_dec_rval_t xer_decode(
    const asn_codec_ctx_t *opt_codec_ctx,
    const asn_TYPE_descriptor_t *type_descriptor,
    void **struct_ptr_ptr,/* Pointer to a target structure's ptr */
    const void *buffer,   /* Data to be decoded */
    size_t size           /* Size of data buffer */
);
```

Description

Decode the BASIC-XER and CANONICAL-XER (XML Encoding Rules) encoding, as defined by ITU-T X.693.

*Consider using a more generic function **`asn_decode(ATS_BASIC_XER)`**.*

Return values

Upon unsuccessful termination, the **`*struct_ptr_ptr`** may contain partially decoded data. This data may be useful for debugging (such as by using **`asn_fprint()`**). Don't forget to discard the unused partially decoded data by calling **`ASN_STRUCT_FREE()`** or **`ASN_STRUCT_RESET()`**.

The function returns a compound structure:

```
typedef struct {
    enum {
        RC_OK,          /* Decoded successfully */
        RC_WMORE,       /* More data expected, call again */
        RC_FAIL         /* Failure to decode data */
    } code;             /* Result code */
    size_t consumed;    /* Number of bytes consumed */
} asn_dec_rval_t;
```

The **`.code`** member specifies the decoding outcome.

RC_OK	Decoded successfully and completely
RC_WMORE	More data expected, call again
RC_FAIL	Failed for good

The **.consumed** member specifies the amount of **buffer** data that was used during parsing, irrespectively of the **.code**.

The **.consumed** value is in bytes.

Restartability

The **xer_decode()** function is restartable (stream-oriented). That means that in case the buffer has less data than expected, the decoder will process whatever is available and ask for more data to be provided using the RC_WMORE return **.code**.

Note that in the RC_WMORE case the decoder may have processed less data than it is available in the buffer, which means that you must be able to arrange the next buffer to contain the unprocessed part of the previous buffer.

3.22 xer_encode()

Synopsis

```
enum xer_encoder_flags_e {
    /* Mode of encoding */
    XER_F_BASIC     = 0x01, /* BASIC-XER (pretty-printing) */
    XER_F_CANONICAL = 0x02  /* Canonical XER (strict rules) */
};
asn_enc_rval_t xer_encode(
    const asn_TYPE_descriptor_t *type_descriptor,
    const void *structure_to_encode,
    enum xer_encoder_flags_e xer_flags,
    asn_app_consume_bytes_f *callback,
    void *callback_key);
```

Description

The **`xer_encode()`** function serializes the given **`structure_to_encode`** using the BASIC-XER or CANONICAL-XER transfer syntax (XML Encoding Rules, ITU-T X.693).

During serialization, a user-specified **`callback`** is invoked zero or more times with bytes of data to add to the output stream (if any), and the **`callback_key`**. The signature for the callback is as follows:

```
typedef int(asn_app_consume_bytes_f)(const void *buffer, size_t
   size, void *callback_key);
```

*Consider using a more generic function **`asn_encode()`** with `ATS_BASIC_XER` or `ATS_CANONICAL_XER` transfer syntax option.*

Return values

The function returns a compound structure:

```
   typedef struct {
       ssize_t encoded;
       const asn_TYPE_descriptor_t *failed_type;
       const void *structure_ptr;
   } asn_enc_rval_t;
```

In case of unsuccessful encoding, the **.encoded** member is set to -1 and the other members of the compound structure point to where the encoding has failed to proceed further.

In case encoding is successful, the **.encoded** member specifies the size of the serialized output.

The serialized output size is returned in **bytes**.

Example

```
static int
save_to_file(const void *data, size_t size, void *key) {
    FILE *fp = key;
    return (fwrite(data, 1, size, fp) == size) ? 0 : -1;
}

Rectangle_t *rect = ...;
FILE *fp = ...;
asn_enc_rval_t er;
er = xer_encode(&asn_DEF_Rectangle, rect, XER_F_CANONICAL, save_to_file, fp);
if(er.encoded == -1) {
   fprintf(stderr, "Failed to encode %s\n", asn_DEF_Rectangle.name);
} else {
   fprintf(stderr, "%s encoded in %zd bytes\n", asn_DEF_Rectangle.name, er.encoded);
}
```

See also

xer_fprint() at page 57.

3.23 xer_fprint()

Synopsis

```
int xer_fprint(FILE *stream,    /* Destination file */
    const asn_TYPE_descriptor_t *type_descriptor,
    const void *struct_ptr      /* Structure to be printed */
);
```

Description

The **xer_fprint()** function outputs XML-based serialization of the given structure into the file stream specified by **stream** pointer.

The output conforms to a BASIC-XER transfer syntax, as defined by ITU-T X.693.

Return values

0 XML output was successfully made

-1 Error printing out the structure

Since the **xer_fprint()** function attempts to produce a conforming output, it will likely break on partial structures by writing incomplete data to the output stream and returning -1. This makes it less suitable for debugging complex cases than **asn_fprint()**.

Example

```
Rectangle_t *rect = ...;
xer_fprint(stdout, &asn_DEF_Rectangle, rect);
```

See also

asn_fprint() at page 32.

Chapter 4

CBOR API usage examples

4.1 cbor_decode()

Synopsis

```
asn_dec_rval_t cbor_decode(
    const asn_codec_ctx_t *opt_codec_ctx,   /* Optional codec
       context */
    const asn_TYPE_descriptor_t *type_descriptor,
    void **struct_ptr,  /* Pointer to a target structure's pointer
       */
    const void *buffer, /* Data to be decoded */
    size_t size         /* Size of data buffer */
);
```

Description

The **cbor_decode()** function decodes a CBOR-encoded (Concise Binary Object Representation, RFC 8949) data stream into the target ASN.1 structure.

CBOR provides a compact, self-describing binary encoding format. The decoder follows canonical CBOR rules: only definite-length items are accepted. Integer values are decoded from CBOR major type 0 (unsigned) and major type 1 (negative), with bignum tags (2 and 3) used for values that exceed the 64-bit range. SEQUENCE and SET types are decoded from CBOR maps, SEQUENCE OF and SET OF from CBOR arrays, and CHOICE from a single-entry CBOR map.

The decoder is *tag-transparent*: any number of leading CBOR tag headers (major type 6, RFC 8949 §3.4) are silently skipped before the underlying value is decoded. This means that tagged CBOR data items produced by other implementations—such as self-described CBOR (tag 55799), epoch-based timestamps (tag 1), or URI-annotated byte strings (tag 32)—are accepted without changes to the application code.

Return values

RC_OK	Structure decoded successfully
RC_WMORE	More data needed to decode
RC_FAIL	Decoding failed

4.2 cbor_encode()

Synopsis

```
asn_enc_rval_t cbor_encode(
    const asn_TYPE_descriptor_t *type_descriptor,
    const void *struct_ptr,             /* Structure to be encoded
       */
    asn_app_consume_bytes_f *consume_bytes_cb, /* Callback */
    void *app_key                       /* Arbitrary callback
       argument */
);
```

Description

The **`cbor_encode()`** function serializes the given structure into a canonical CBOR (Concise Binary Object Representation, RFC 8949) byte stream.

The encoder produces canonical (DER-like) output: integers use the shortest possible CBOR encoding, all lengths are definite, and SEQUENCE fields are encoded in definition order. Absent optional and default-valued fields are skipped so that the CBOR map header count always matches the number of encoded key/value pairs.

Return values

er.encoded ≥ 0	Number of bytes produced
er.encoded $= -1$	Encoding failed; **`er.failed_type`** identifies the culprit

4.3 CBOR Tags

Overview

CBOR tags (RFC 8949 §3.4, major type 6) are optional semantic annotations that precede a data item. A tag consists of a tag number—an unsigned integer identifying the semantic type—followed by exactly one tagged data item. Tags do not alter the binary representation of the wrapped value; they add out-of-band meaning for applications and interchange.

The complete list of registered tag numbers is maintained by IANA:

https://www.iana.org/assignments/cbor-tags/

Encoding with tags

The **cbor_support.h** header defines symbolic constants for frequently used tags:

Constant	Tag#	Semantics (RFC 8949 / IANA)
CBOR_TAG_DATETIME_STRING	0	Standard date/time text string
CBOR_TAG_EPOCH_DATETIME	1	Epoch-based date/time number
CBOR_TAG_POSINT_BIGNUM	2	Positive bignum (used internally)
CBOR_TAG_NEGINT_BIGNUM	3	Negative bignum (used internally)
CBOR_TAG_DECIMAL_FRACTION	4	Decimal fraction
CBOR_TAG_BIGFLOAT	5	Bigfloat
CBOR_TAG_BASE64URL	21	Expected base64url encoding hint
CBOR_TAG_BASE64	22	Expected base64 encoding hint
CBOR_TAG_BASE16	23	Expected base16 encoding hint
CBOR_TAG_ENCODED_CBOR	24	Encoded CBOR data item
CBOR_TAG_URI	32	URI text string
CBOR_TAG_REGEX	35	Regular expression text string
CBOR_TAG_UUID	37	Binary UUID (16 bytes)
CBOR_TAG_NETWORK_ADDR	260	IPv4/IPv6/MAC network address
CBOR_TAG_SELF_DESCRIBED	55799	Self-described CBOR marker

To prepend a CBOR tag to any encoded value, call **cbor_encode_tag()** before encoding the value. The following example emits the self-described CBOR marker (tag 55799) followed by an INTEGER value:

```
#include <cbor_support.h>
#include <cbor_encoder.h>
#include <INTEGER.h>
```

```
static int my_write(const void *data, size_t size, void *key) {
    /* write data to output stream */
    return fwrite(data, 1, size, (FILE *)key) == size ? 0 : -1;
}

void encode_tagged_integer(INTEGER_t *val, FILE *out) {
    /* Emit self-described CBOR tag (55799) first */
    cbor_encode_tag(CBOR_TAG_SELF_DESCRIBED, my_write, out);
    /* Then encode the INTEGER value */
    cbor_encode(&asn_DEF_INTEGER, val, my_write, out);
}
```

Arbitrary tag numbers not listed above can be passed directly to **`cbor_encode_tag()`** as a **`uint64_t`**.

Decoding tagged data

All asn1c CBOR decoders are *tag-transparent*: any number of leading tag headers are automatically consumed before the underlying value is decoded. No special handling is needed in application code; tagged and untagged data items are decoded identically.

The helper function **`cbor_skip_tags()`** (declared in **`cbor_support.h`**) is also available for applications that inspect raw CBOR buffers directly:

```
#include <cbor_support.h>

/*
 * Returns the number of bytes occupied by leading tag headers,
 * or -1 on error.  After the call, buf + n points to the
 * first non-tag byte.
 */
ssize_t cbor_skip_tags(const uint8_t *buf, size_t size);
```

Applications that need to skip a complete raw CBOR item can call **`cbor_skip_item()`**. Code that runs as part of a decoder and has an **`asn_codec_ctx_t`** should call **`cbor_skip_item_with_ct`** instead, so recursive arrays, maps, and tags are bounded by the same stack limit as normal ASN.1 decoding:

```
#include <cbor_support.h>
```

```
/*
 * Returns the number of bytes occupied by a complete CBOR item,
 * or -1 on error, truncation, or decoder stack limit.
 */
ssize_t cbor_skip_item_with_ctx(const asn_codec_ctx_t
   *opt_codec_ctx,
                                const uint8_t *buf, size_t size);
ssize_t cbor_skip_item(const uint8_t *buf, size_t size);
```

Bignum tags

CBOR tags 2 and 3 are treated specially by the INTEGER decoder. They encode arbitrarily large positive (tag 2) and negative (tag 3) integers as byte strings, allowing ASN.1 INTEGER values that exceed the 64-bit signed range to be represented in CBOR. These tags are emitted automatically by the INTEGER encoder when the value requires more than 64 bits.

Chapter 5

API usage examples

Let's start with including the necessary header files into your application. Normally it is enough to include the header file of the main PDU type. For our *Rectangle* module, including the *Rectangle.h* file is sufficient:

```
#include <Rectangle.h>
```

The header files defines a C structure corresponding to the ASN.1 definition of a rectangle and the declaration of the ASN.1 *type descriptor*. A type descriptor is a special globally accessible object which is used as an argument to most of the API functions provided by the ASN.1 codec. A type descriptor starts with *asn_DEF_*.... For example, here is the code which frees the Rectangle_t structure:

```
Rectangle_t *rect = ...;

ASN_STRUCT_FREE(asn_DEF_Rectangle, rect);
```

This code defines a *rect* pointer which points to the Rectangle_t structure which needs to be freed. The second line uses a generic **ASN_STRUCT_FREE()** macro which invokes the memory deallocation routine created specifically for this Rectangle_t structure. The *asn_DEF_Rectangle* is the type descriptor which holds a collection of routines and operations defined for the Rectangle_t structure.

5.1 Generic encoders and decoders

Before we start describing specific encoders and decoders, let's step back a little and check out a simple high level way.

The asn1c runtime supplies (see *asn_application.h*) two sets of high level functions, **asn_encode*** and **asn_decode***, which take a transfer syntax selector as an argument. The transfer syntax selector is defined as this:

```
/* A selection of ASN.1 transfer syntaxes for generic encoders and decoders. */
enum asn_transfer_syntax {
    ATS_INVALID = 0,
    ATS_NONSTANDARD_PLAINTEXT,
    ATS_RANDOM,
    ATS_BER,
    ATS_DER,
    ATS_CER,
    ATS_BASIC_OER,
    ATS_CANONICAL_OER,
    ATS_UNALIGNED_BASIC_PER,
    ATS_UNALIGNED_CANONICAL_PER,
    ATS_ALIGNED_BASIC_PER,
    ATS_ALIGNED_CANONICAL_PER,
    ATS_BASIC_XER,
    ATS_CANONICAL_XER,
    ATS_JER,
    ATS_JER_MINIFIED,
    ATS_CBOR,
};
```

Using this encoding selector, encoding and decoding becomes very generic:

Encoding:

```
uint8_t buffer[128];
size_t buf_size = sizeof(buffer);
asn_enc_rval_t er;

er = asn_encode_to_buffer(0, ATS_DER, &asn_DEF_Rectangle, buffer, buf_size);

if(er.encoded > buf_size) {
    fprintf(stderr, "Buffer of size %zu is too small for %s, need %zu\n",
        buf_size, asn_DEF_Rectangle.name, er.encoded);
}
```

Decoding:

```
Rectangle_t *rect = 0;    /* Note this 0[1]! */

... = asn_decode(0, ATS_BER, &asn_DEF_Rectangle, (void **)&rect, buffer, buf_size);
```

5.2 Encoding CER

The callback API is `cer_encode()`; use `cer_encode_to_buffer()` with caller-owned storage. The generic `asn_encode()` and `asn_encode_to_buffer()` functions select the

[1] Forgetting to properly initialize the pointer to a destination structure is a major source of support requests.

same encoder with ATS_CER. For example, encode a value into a caller-provided buffer:

```
Rectangle_t *rect = ...;
uint8_t buffer[4096];
asn_enc_rval_t er = cer_encode_to_buffer(
    0, &asn_DEF_Rectangle, rect, buffer, sizeof(buffer));
if(er.encoded < 0) {
    /* Check errno for the reason encoding failed. */
}
```

To send encoded bytes to a callback, use `cer_encode()`:

```
static int write_bytes(const void *data, size_t size, void *key) {
    FILE *stream = (FILE *)key;
    return fwrite(data, 1, size, stream) == size ? 0 : -1;
}
asn_enc_rval_t er = cer_encode(
    0, &asn_DEF_Rectangle, rect, write_bytes, stdout);
```

The generic form selects the same CER encoder:

```
asn_enc_rval_t er = asn_encode_to_buffer(
    0, ATS_CER, &asn_DEF_Rectangle, rect, buffer, sizeof(buffer));
```

The direct `cer_encode_to_buffer()` follows the direct DER encoder contract and reports failure when the buffer is too small. The generic `asn_encode_to_buffer()` reports the complete required size in `.encoded`; when the buffer is short, only a prefix fits.

Generated converter programs accept `-icer` for BER-backed input and `-ocer` for CER output:

```
./converter-example -iber -ocer input.ber > output.cer
```

CER input is decoded by the shared BER decoder. The ATS_CER selector does not independently validate canonical CER form. C support is generated by default; `-no-gen-CER` omits CER encoding support, while `-no-gen-BER` also disables its BER-family dependencies. The ASN_DISABLE_CER_SUPPORT build macro disables the encoder while leaving BER-backed CER input available. Opaque ANY, unresolved standalone open types, and decimal or noncanonical raw REAL values are not encoded; raw REAL accepts only zero, special values, and minimal base-2 forms with exponents of at most eight octets. `UTCTime` is normalized to UTC seconds. `GeneralizedTime` requires an explicit zone and supports at most nine fractional-second digits; comma input is normalized to a period. The descriptor walker applies finite output, scratch, work, collection, recursive-traversal, and tag-chain limits. An additional explicit member tag counts against the tag-chain cap; see the binary codec support audit for defaults and the type boundary.

Constructed character strings retain their outer type tag and encode their fragments with universal OCTET STRING tag 4. BIT STRING fragments use universal tag 3. BER input preserves legacy opaque ANY containers while checking their lengths, nesting, and EOC markers.

Typed string wrappers require one child value; this compatibility does not imply canonical open-type validation.

5.3 Generating Rust code

The `--rust` option selects a standalone Cargo project with a copied, safe Rust runtime. The compiler still parses and fixes ASN.1 in C; Cargo builds the generated Rust package. The current slice supports UPER encoding and decoding, plus CER encoding, for top-level BOOLEAN, NULL, and finite, nonextensible INTEGER ranges fitting signed 64-bit values.

```
RustDemo DEFINITIONS ::= BEGIN
Flag ::= BOOLEAN
Empty ::= NULL
Count ::= INTEGER (-10..10)
END
```

Generate and build the package offline:

```
asn1c --rust -D generated RustDemo.asn1
cargo check --offline --manifest-path generated/Cargo.toml
cargo test --offline --manifest-path generated/Cargo.toml
```

The generated module provides `encode_cer()` for all three values. For UPER it provides `encode_uper()` and `decode_uper()`. For example:

```
use asn1c_generated::rust_demo::{Count, Empty, Flag};
use asn1c_runtime::Limits;

let limits = Limits::default();
let flag = Flag::new(true);
let cer = flag.encode_cer(&limits)?;
assert_eq!(cer.bytes, [0x01, 0x01, 0xff]);

let uper = flag.encode_uper(&limits)?;
let decoded = Flag::decode_uper(&uper.bytes, &limits)?;
assert!(decoded.value.value());

let number = Count::new(-3)?;
let number_cer = number.encode_cer(&limits)?;
let empty_cer = Empty.encode_cer(&limits)?;
```

To generate only CER methods, pass `--rust -no-gen-UPER`. The default output directory is `asn1c-rust`; use `-D` to choose another. The generator records its files in an ownership manifest and stages output before publishing it. It rejects unrelated existing files and

symlinks in the destination. Use a fresh directory after adding application code or building with Cargo, which creates its own `target/` directory.

Other ASN.1 forms and Rust codecs are not yet supported; unsupported requests fail before output is published. Rust has no CER decoder. C ATS_CER input remains BER-backed and does not validate canonical CER form. The Rust package uses the Rust 2021 edition and has no third-party runtime dependencies. No minimum Rust compiler version is claimed.

5.4 Decoding BER

The Basic Encoding Rules describe the most widely used (by the ASN.1 community) way to encode and decode a given structure in a machine-independent way. Several other encoding rules (CER, DER) define a more restrictive versions of BER, so the generic BER parser is also capable of decoding the data encoded by the CER and DER encoders. The opposite is not true.

The ASN.1 compiler provides the generic BER decoder which is capable of decoding BER, CER and DER encoded data.

The decoder is restartable (stream-oriented). That means that in case the buffer has less data than expected, the decoder will process whatever is available and ask for more data to be provided using the RC_WMORE return **`.code`**.

Note that in the RC_WMORE case the decoder may have processed less data than it is available in the buffer, which means that you must be able to arrange the next buffer to contain the unprocessed part of the previous buffer.

Suppose, you have two buffers of encoded data: 100 bytes and 200 bytes.

- You can concatenate these buffers and feed the BER decoder with 300 bytes of data, or
- You can feed it the first buffer of 100 bytes of data, realize that the ber_decoder consumed only 95 bytes from it and later feed the decoder with 205 bytes buffer which consists of 5 unprocessed bytes from the first buffer and the additional 200 bytes from the second buffer.

This is not as convenient as it could be (the BER encoder could consume the whole 100 bytes and keep these 5 bytes in some temporary storage), but in case of existing stream based processing it might actually fit well into existing algorithm. Suggestions are welcome.

Here is the example of BER decoding of a simple structure:

```
Rectangle_t *
simple_deserializer(const void *buffer, size_t buf_size) {
```

```
        asn_dec_rval_t rval;
        Rectangle_t *rect = 0;    /* Note this 0[1]! */

        rval = asn_DEF_Rectangle.op->ber_decoder(0,
              &asn_DEF_Rectangle,
              (void **)&rect,    /* Decoder changes the pointer */
              buffer, buf_size, 0);

        if(rval.code == RC_OK) {
            return rect;          /* Decoding succeeded */
        } else {
            /* Free the partially decoded rectangle */
            ASN_STRUCT_FREE(asn_DEF_Rectangle, rect);
            return 0;
        }
    }
```

The code above defines a function, *simple_deserializer*, which takes a buffer and its length and is expected to return a pointer to the Rectangle_t structure. Inside, it tries to convert the bytes passed into the target structure (rect) using the BER decoder and returns the rect pointer afterwards. If the structure cannot be deserialized, it frees the memory which might be left allocated by the unfinished *ber_decoder* routine and returns 0 (no data). (This **freeing is necessary** because the ber_decoder is a restartable procedure, and may fail just because there is more data needs to be provided before decoding could be finalized). The code above obviously does not take into account the way the *ber_decoder()* failed, so the freeing is necessary because the part of the buffer may already be decoded into the structure by the time something goes wrong.

A little less wordy would be to invoke a globally available *ber_decode()* function instead of dereferencing the asn_DEF_Rectangle type descriptor:

```
rval = ber_decode(0, &asn_DEF_Rectangle, (void **)&rect, buffer,
   buf_size);
```

Note that the initial (asn_DEF_Rectangle.op->ber_decoder) reference is gone, and also the last argument (0) is no longer necessary.

These two ways of BER decoder invocations are fully equivalent.

The BER de*coder* may fail because of (*the following RC_... codes are defined in ber_decoder.h*):

[1] Forgetting to properly initialize the pointer to a destination structure is a major source of support requests.

- RC_WMORE: There is more data expected than it is provided (stream mode continuation feature);
- RC_FAIL: General failure to decode the buffer;
- ... other codes may be defined as well.

Together with the return code (.code) the asn_dec_rval_t type contains the number of bytes which is consumed from the buffer. In the previous hypothetical example of two buffers (of 100 and 200 bytes), the first call to ber_decode() would return with .code = RC_WMORE and .consumed = 95. The .consumed field of the BER decoder return value is **always** valid, even if the decoder succeeds or fails with any other return code.

Look into ber_decoder.h for the precise definition of ber_decode() and related types.

5.5 Encoding DER

The Distinguished Encoding Rules is the *canonical* variant of BER encoding rules. The DER is best suited to encode the structures where all the lengths are known beforehand. This is probably exactly how you want to encode: either after a BER decoding or after a manual fill-up, the target structure contains the data which size is implicitly known before encoding. Among other uses, the DER encoding is used to encode X.509 certificates.

As with BER decoder, the DER encoder may be invoked either directly from the ASN.1 type descriptor (asn_DEF_Rectangle) or from the stand-alone function, which is somewhat simpler:

```
/*
 * This is the serializer itself.
 * It supplies the DER encoder with the
 * pointer to the custom output function.
 */
ssize_t
simple_serializer(FILE *ostream, Rectangle_t *rect) {
    asn_enc_rval_t er;  /* Encoder return value */

    er = der_encode(&asn_DEF_Rect, rect, write_stream, ostream);
    if(er.encoded == -1) {
        fprintf(stderr, "Cannot encode %s: %s\n",
            er.failed_type->name, strerror(errno));
```

```
            return -1;
        } else {
            /* Return the number of bytes */
            return er.encoded;
        }
    }
```

As you see, the DER encoder does not write into some sort of buffer. It just invokes the custom function (possible, multiple times) which would save the data into appropriate storage. The optional argument *app_key* is opaque for the DER encoder code and just used by *write_stream()* as the pointer to the appropriate output stream to be used.

If the custom write function is not given (passed as 0), then the DER encoder will essentially do the same thing (i. e., encode the data) but no callbacks will be invoked (so the data goes nowhere). It may prove useful to determine the size of the structure's encoding before actually doing the encoding[1].

Look into der_encoder.h for the precise definition of der_encode() and related types.

5.6 Encoding XER

The XER stands for XML Encoding Rules, where XML, in turn, is eXtensible Markup Language, a text-based format for information exchange. The encoder routine API comes in two flavors: stdio-based and callback-based. With the callback-based encoder, the encoding process is very similar to the DER one, described in section 5.5 on the previous page. The following example uses the definition of write_stream() from up there.

```
/*
 * This procedure generates an XML document
 * by invoking the XER encoder.
 * NOTE: Do not copy this code verbatim!
 *       If the stdio output is necessary,
 *       use the xer_fprint() procedure instead.
 *       See section 5.9 on page 73.
 */
int
print_as_XML(FILE *ostream, Rectangle_t *rect) {
    asn_enc_rval_t er;  /* Encoder return value */
```

[1] It is actually faster too: the encoder might skip over some computations which aren't important for the size determination.

```
    er = xer_encode(&asn_DEF_Rectangle, rect,
        XER_F_BASIC, /* BASIC-XER or CANONICAL-XER */
        write_stream, ostream);

    return (er.encoded == -1) ? -1 : 0;
}
```

Look into xer_encoder.h for the precise definition of xer_encode() and related types.

See section 5.9 on the next page for the example of stdio-based XML encoder and other pretty-printing suggestions.

5.7 Decoding XER

The data encoded using the XER rules can be subsequently decoded using the xer_decode() API call:

```
Rectangle_t *
XML_to_Rectangle(const void *buffer, size_t buf_size) {
    asn_dec_rval_t rval;
    Rectangle_t *rect = 0;    /* Note this 0[1]! */

    rval = xer_decode(0, &asn_DEF_Rectangle, (void **)&rect,
       buffer, buf_size);

    if(rval.code == RC_OK) {
        return rect;          /* Decoding succeeded */
    } else {
        /* Free partially decoded rect */
        ASN_STRUCT_FREE(asn_DEF_Rectangle, rect);
        return 0;
    }
}
```

The decoder takes both BASIC-XER and CANONICAL-XER encodings.

The decoder shares its data consumption properties with BER decoder; please read the section 5.4 on page 68 to know more.

Look into xer_decoder.h for the precise definition of xer_decode() and related types.

[1] Forgetting to properly initialize the pointer to a destination structure is a major source of support requests.

5.8 Validating the target structure

Sometimes the target structure needs to be validated. For example, if the structure was created by the application (as opposed to being decoded from some external source), some important information required by the ASN.1 specification might be missing. On the other hand, the successful decoding of the data from some external source does not necessarily mean that the data is fully valid either. It might well be the case that the specification describes some subtype constraints that were not taken into account during decoding, and it would actually be useful to perform the last check when the data is ready to be encoded or when the data has just been decoded to ensure its validity according to some stricter rules.

The **`asn_check_constraints()`** function checks the type for various implicit and explicit constraints. It is recommended to use the **`asn_check_constraints()`** function after each decoding and before each encoding.

5.9 Printing the target structure

To print out the structure for debugging purposes, use the **`asn_fprint()`** function:

```
asn_fprint(stdout, &asn_DEF_Rectangle, rect);
```

A practical alternative to this custom format printing is to serialize the structure into XML. The default BASIC-XER encoder performs reasonable formatting for the output to be both useful and human readable. Use the **`xer_fprint()`** function:

```
xer_fprint(stdout, &asn_DEF_Rectangle, rect);
```

See section 5.6 on page 71 for XML-related details.

5.10 Freeing the target structure

Freeing the structure is slightly more complex than it may seem. When the ASN.1 structure is freed, all the members of the structure and their submembers are recursively freed as well. The ASN_STRUCT_FREE() macro helps with that.

But it might not always be feasible to free the whole structure. In the following example, the application programmer defines a custom structure with one ASN.1-derived member (rect).

```
struct my_figure {       /* The custom structure */
```

```
    int flags;          /* <some custom member> */
    /* The type is generated by the ASN.1 compiler */
    Rectangle_t rect;
    /* other members of the structure */
};
```

This member is not a reference to the Rectangle_t, but an in-place inclusion of the Rectangle_t structure. If there's a need to free the **rect** member, the usual procedure of freeing everything must not be applied to the **&rect** pointer itself, because it does not point to the beginning of memory block allocated by the memory allocation routine, but instead lies within a block allocated for the my_figure structure.

To solve this problem, in addition to ASN_STRUCT_FREE() macro, the asn1c skeletons define the ASN_STRUCT_RESET() macro which doesn't free the passed pointer and instead resets the structure into the clean and safe state.

```
/* 1. Rectangle_t is defined within my_figure */
struct my_figure {
    Rectangle_t rect;
} *mf = ...;
/*
 * Freeing the Rectangle_t
 * without freeing the mf->rect area.
 */
ASN_STRUCT_RESET(asn_DEF_Rectangle, &mf->rect);

/* 2. Rectangle_t is a stand-alone pointer */
Rectangle_t *rect = ...;
/*
 * Freeing the Rectangle_t
 * and freeing the rect pointer.
 */
ASN_STRUCT_FREE(asn_DEF_Rectangle, rect);
```

It is safe to invoke both macros with the target structure pointer set to 0 (NULL). In this case, the function will do nothing.

Chapter 6

Advanced Features

6.1 Unknown extension decoding

Upgrade warning. Decoding behavior for unknown extensions has changed from "fail" to "skip/relay". Evaluate the impact on your application before upgrading. To restore the previous strict behavior, use the build definition shown below.

The forward-compatible default applies when an older schema decodes a value produced using a newer version of an extensible ASN.1 type:

- An unknown UPER or OER **`CHOICE`** extension alternative is consumed and skipped. The decoder returns **`RC_OK`**, with no local alternative selected.
- An unknown UPER **`ENUMERATED`** extension addition is represented in the reserved **`LONG_MAX - extension_index`** range. Re-encoding that value with UPER relays the original extension index.

The change does not make truncated or structurally malformed encodings valid. It does mean that an unknown, structurally valid extension no longer produces **`RC_FAIL`**. Applications that used that failure as an implicit input- validation gate, or whose state machines treat every decode failure as a protocol error, should review their behavior before upgrading.

To retain strict rejection, compile the decoder skeleton sources with:

```
cc -DASN_REJECT_UNKNOWN_EXTENSIONS ...
```

Apply the definition consistently to all generated/runtime skeleton objects and perform a clean rebuild. Defining the macro only in application code does not change decoder objects in an already-built runtime library. Enabling strict rejection forfeits forward compatibility for these extension additions.

6.2 ENCODING-CONTROL Directives

The ENCODING-CONTROL directive allows you to customize how specific types are encoded and decoded in XML Encoding Rules (XER). This feature is particularly useful when you need to control the representation of binary data in XML documents.

Syntax

The ENCODING-CONTROL directive is specified within an ASN.1 module using the following syntax:

```
ENCODING-CONTROL <encoding-reference>
    <type-name> <type-constraint> ::= <encoding-format>
    ...
END
```

Where:

- **encoding-reference**: The encoding rules to which the directive applies (e.g., XER)
- **type-name**: The name of the type being controlled
- **type-constraint**: The ASN.1 type (e.g., `OCTET STRING`)
- **encoding-format**: The desired encoding format (see below)

Supported Encoding Formats

The following encoding formats are currently supported for OCTET STRING types:

- **hexadecimal**: Encodes binary data as hexadecimal digits (0-9, A-F)
- **base64**: Encodes binary data using Base64 encoding (default behavior)
- **utf8**: Treats the OCTET STRING as UTF-8 text and encodes it directly

Example

Consider a PKI application that needs to encode certificate extensions:

```
CertificateModule DEFINITIONS AUTOMATIC TAGS ::= BEGIN

Extension ::= SEQUENCE {
    extnID      OBJECT IDENTIFIER,
    critical    BOOLEAN DEFAULT FALSE,
    extnValue   ExtensionValue
}

ExtensionValue ::= OCTET STRING

ENCODING-CONTROL XER
    ExtensionValue OCTET STRING ::= hexadecimal
END

END
```

With this directive, when encoding an Extension to XER format, the `extnValue` field will be represented as hexadecimal:

```
<Extension>
    <extnID>2.5.29.19</extnID>
    <critical>true</critical>
    <extnValue>30030101FF</extnValue>
</Extension>
```

Without the ENCODING-CONTROL directive, the same data would be Base64-encoded by default.

Compilation

When compiling ASN.1 modules containing ENCODING-CONTROL directives, the compiler generates custom XER encoder and decoder functions for the affected types:

```
asn1c -no-gen-example CertificateModule.asn1
```

The compiler will output a note indicating which encoding controls were applied:

```
NOTE: ENCODING-CONTROL XER at CertificateModule.asn1:15 with 1
   directive(s)
NOTE: Applied 1 encoding control directive(s) in module
   CertificateModule
```

Runtime Behavior

At runtime, types with ENCODING-CONTROL directives use custom encoder and decoder functions:

- **hexadecimal**: Binary data `{0xDE, 0xAD, 0xBE, 0xEF}` encodes as `DEADBEEF`
- **utf8**: Text data "Hello World" encodes directly without Base64 wrapping
- **base64**: Standard Base64 encoding (unchanged from default behavior)

XER Decoding Support

The XER decoder has been enhanced to support X.693 hexadecimal string notation using the `H'...'` syntax:

```
<Extension>
    <extnValue>H'30030101FF'</extnValue>
</Extension>
```

This notation is automatically recognized and decoded by the **`OCTET_STRING_decode_xer()`** function through the enhanced **`OCTET_STRING__convert_auto()`** converter.

Empty OPTIONAL Fields in XER

By default, the XER decoder treats empty XML tags (e.g., `<field></field>` or `<field/>`) for OPTIONAL fields as invalid, causing decoding failures. This differs from BER/DER and PER encodings, which simply omit absent optional fields.

To enable lenient handling of empty OPTIONAL fields in XER, compile the generated code with the `XER_ALLOW_EMPTY_OPTIONALS` preprocessor flag:

```
gcc -DXER_ALLOW_EMPTY_OPTIONALS -c *.c
```

When this flag is enabled, empty tags for OPTIONAL fields are treated as absent rather than invalid. For example:

```
<TestSequence>
    <mandatoryInt>42</mandatoryInt>
    <optionalInt></optionalInt>    <!-- Treated as absent -->
    <optionalString/>              <!-- Treated as absent -->
</TestSequence>
```

In the above example, both `optionalInt` and `optionalString` will be decoded as absent (NULL pointers), not as present with invalid/empty values.

Important notes:

- Empty tags for mandatory fields still cause decoding failures, preserving data integrity
- This feature only affects XER decoding; other encoding rules are unaffected
- The feature is disabled by default to maintain strict XML validation
- Both self-closing tags (`<field/>`) and separate empty tags (`<field></field>`) are supported

See also

For implementation details, see:

- `libasn1compiler/asn1c_encoding.c` – Encoding control application logic
- `skeletons/OCTET_STRING_xer.c` – Runtime XER encoding/decoding functions
- `skeletons/constr_SEQUENCE_xer.c` – Empty optional field detection
- `skeletons/asn_internal.h` – XER_ALLOW_EMPTY_OPTIONALS flag definition
- `ENCODING_CONTROL_STATUS.md` – Complete implementation status

6.3 Circular Dependency Handling

Circular dependencies occur when two or more ASN.1 types reference each other, directly or indirectly, creating a cycle in the type dependency graph. The asn1c compiler includes sophisticated mechanisms to detect and resolve these dependencies.

Detection

The compiler analyzes the type dependency graph to identify circular references. For example:

```
Node ::= SEQUENCE {
    value   INTEGER,
    next    Node OPTIONAL     -- References itself
}
```

```
TypeA ::= SEQUENCE {
    field-b TypeB OPTIONAL
}

TypeB ::= SEQUENCE {
    field-a TypeA OPTIONAL
}
```

In the first example, `Node` has a self-reference. In the second, `TypeA` and `TypeB` reference each other, creating a circular dependency.

Resolution Strategies

The compiler employs several strategies to break circular dependencies:

1. Automatic Pointer Indirection

When a circular dependency is detected, the compiler automatically converts certain type members to pointers. This is the most common resolution mechanism.

2. The `-findirect-choice` Flag

This flag forces CHOICE type members to be compiled as indirect pointers:

```
asn1c -findirect-choice module.asn1
```

This is particularly useful for:

- Complex specifications with many CHOICE types (e.g., F1AP, NGAP)
- Information Object Class OPEN_TYPE fields
- Breaking circular dependencies involving CHOICE constructs

Example: F1AP Compilation

The F1AP specification is a complex 3GPP protocol with extensive circular dependencies. To successfully compile F1AP:

```
asn1c -findirect-choice F1AP-PDU-Descriptions.asn1 \textbackslash
      F1AP-IEs.asn1 F1AP-Containers.asn1 F1AP-Constants.asn1
```

This approach:

1. Forces CHOICE members to be indirect pointers
2. Breaks circular dependencies involving CHOICE types
3. Handles Information Object Class tables correctly

Generated Code

For a type with circular dependencies, the compiler converts members to pointers:

```
/* Original ASN.1:
 * Node ::= SEQUENCE {
 *     value   INTEGER,
 *     next    Node OPTIONAL
 * }
 */

/* Generated C: */
typedef struct Node {
    long      value;
    struct Node  *next;  /* OPTIONAL, pointer to break cycle */
} Node_t;
```

Without proper handling, the compiler might fail with ”circular dependency” errors, or generate code that causes C compiler errors.

Backward Compatibility

The circular dependency detection maintains backward compatibility:

- Default behavior: only actual cycles trigger pointer indirection
- The `-findirect-choice` flag must be explicitly enabled for CHOICE indirection
- Existing ASN.1 modules compile without modification
- Generated code remains compatible with existing applications

See also

For implementation details:

- `libasn1compiler/asn1c_constraint.c` – Circular dependency detection
- `libasn1fix/asn1fix_crange.c` – Constraint range fixing with cycle detection

6.4 Information Object Class Support

Information Object Classes (IOCs) are an advanced ASN.1 feature used for defining protocol data structures with type-value relationships. The asn1c compiler provides comprehensive support for IOC tables and their use in type definitions.

What are Information Object Classes?

Information Object Classes define relationships between identifiers (like OBJECT IDENTIFIERs or INTEGERs) and associated types. They are commonly used in telecommunications protocols to define extensible message structures.

```
PROTOCOL-IE ::= CLASS {
    &id                 ProtocolIE-ID UNIQUE,
    &criticality        Criticality,
    &Value,
    &presence           Presence
}
WITH SYNTAX {
    ID                  &id
    CRITICALITY         &criticality
    TYPE                &Value
    PRESENCE            &presence
}

ProtocolIE-Container { PROTOCOL-IE : IEsSetParam } ::=
    SEQUENCE (SIZE (0..maxProtocolIEs)) OF
        ProtocolIE-Field { {IEsSetParam} }

ProtocolIE-Field { PROTOCOL-IE : IEsSetParam } ::= SEQUENCE {
    id                  PROTOCOL-IE.&id          ({IEsSetParam}),
    criticality         PROTOCOL-IE.&criticality
       ({IEsSetParam}{@id}),
    value               PROTOCOL-IE.&Value
       ({IEsSetParam}{@id})
}
```

Code Generation with `-fgen-only-pdu-deps`

The `-fgen-only-pdu-deps` flag generates code only for types that are dependencies of PDU types. This is particularly useful for large specifications:

```
asn1c -pdu=F1AP-PDU -fgen-only-pdu-deps F1AP-*.asn1
```

The compiler performs sophisticated analysis to include all necessary types:

1. **Direct dependencies**: Types directly referenced by PDU types
2. **IOC table references**: Types referenced in Information Object Class tables
3. **Parameterized types**: Specializations of parameterized types
4. **Constraint-based IOC references**: Types referenced through IOC constraints
5. **Transitive dependencies**: All dependencies of included types

IOC Table Traversal

The compiler traverses IOC tables to identify all referenced types:

- Examines each Information Object in the table
- Extracts type references from object fields
- Follows parameterized type instantiations
- Resolves constraint-based type selections
- Marks all discovered types for code generation

Example: F1AP Protocol

F1AP uses extensive IOC tables for protocol IEs (Information Elements):

```
asn1c -pdu=F1AP-PDU -fgen-only-pdu-deps -findirect-choice
   \textbackslash
      F1AP-PDU-Descriptions.asn1 F1AP-IEs.asn1
         F1AP-Containers.asn1
```

This generates code for:

- The F1AP-PDU type itself
- All IEs referenced in F1AP protocol message definitions
- Container types (ProtocolIE-Container, etc.)
- All types referenced in IOC tables for these IEs
- Parameterized type specializations

Without IOC table traversal, many required types would be omitted, resulting in incomplete or non-compilable generated code.

Parameterized Types

IOC-based definitions often use parameterized types:

```
Container { TYPE-CLASS : TypeSet } ::= SEQUENCE (SIZE(1..100)) OF
    Element { {TypeSet} }

Element { TYPE-CLASS : TypeSet } ::= SEQUENCE {
    id      TYPE-CLASS.&id      ({TypeSet}),
    value   TYPE-CLASS.&Value   ({TypeSet}{@id})
}
```

The compiler:

1. Identifies parameterized type instantiations
2. Generates specialized versions for each instantiation
3. Includes all types referenced in the specializations
4. Maintains correct type relationships

OPEN TYPE Handling

When combined with `-findirect-choice`, IOC OPEN_TYPE fields are generated as indirect pointers:

```
/* ASN.1 OPEN TYPE field:
 * value   PROTOCOL-IE.&Value  ({IEsSetParam}{@id})
 */

/* Generated C structure: */
typedef struct ProtocolIE_Field {
    long                    id;
    long                    criticality;
    struct value_OpenType   *value;  /* Indirect pointer */
} ProtocolIE_Field_t;
```

This prevents circular dependencies and improves code organization for complex protocols.

Benefits

Using `-fgen-only-pdu-deps` with IOC support:

- Reduces generated code size (only needed types)
- Faster compilation of large specifications
- Smaller executable binaries
- Easier code review and maintenance
- Complete and correct dependency resolution

Limitations

Current limitations:

- IOC table analysis is best-effort; complex constraint expressions may be incomplete
- Some rare IOC usage patterns may not be fully analyzed
- Generated code should be tested to verify all required types are present

See also

Implementation details:

- `libasn1compiler/asn1c_ioc.c` – IOC table handling

- `libasn1compiler/asn1c_C.c` – Code generation with IOC support
- `libasn1fix/asn1fix_param.c` – Parameterized type handling

6.5 Using the `-fprefix` Option

The `-fprefix` option adds a prefix to all generated type names and filenames, helping to avoid naming conflicts in various scenarios.

System Header Conflicts on Case-Insensitive Filesystems

On case-insensitive filesystems (macOS HFS+, Windows), ASN.1 types can generate files that conflict with system headers. For example, an ASN.1 type named `Time` generates `Time.h`, which can shadow the system header `<time.h>` when compiling with `-I.`.

Problem:

```
asn1c rfc3280.asn1
cc -I. -o myprogram *.c
```

On macOS or Windows, this may produce errors like:

```
warning: non-portable path to file '<Time.h>'
```

Default behavior:

```
asn1c rfc3280.asn1
```

The generated conflicting filename is automatically disambiguated (for example, `asn1c_time.h` instead of `Time.h`).

Optional custom namespace:

```
asn1c -fprefix=ASN1_ rfc3280.asn1
cc -I. -o myprogram *.c
```

This generates `ASN1_Time.h` instead of `Time.h`, avoiding the conflict.

Using `-fprefix` with PDU Options

When using `-fprefix` with `-pdu` options, the prefix is automatically applied to PDU definitions in `converter-example.mk`.

Specific PDU Type

```
asn1c -fprefix=ASN1_ -pdu=Certificate rfc3280.asn1
```

The generated `converter-example.mk` automatically contains:

```
CFLAGS += $(ASN_MODULE_CFLAGS) -DPDU=ASN1_Certificate -I.
```

No manual editing required – the prefix is applied automatically to the PDU definition.

Multiple PDUs with `-pdu=all`

```
asn1c -fprefix=ASN1_ -pdu=all myschema.asn1
```

When using `-pdu=all` or `-pdu=auto`, the compiler:

1. Generates `pdu_collection.c` with all available PDU types
2. Adds helpful comments in `converter-example.mk` explaining how to select a specific PDU
3. Prints an informative message to stderr with usage instructions

Example output:

```
Generated pdu_collection.c
NOTE: Multiple PDU types available. The converter-example
      uses the first PDU by default.
      To decode a specific PDU type, add -DPDU=ASN1_YourTypeName
      to CFLAGS in converter-example.mk
      See pdu_collection.c for the complete list of PDU types.
```

The generated `converter-example.mk` includes:

```
# PDU COLLECTION MODE: Multiple PDU types are available.
# The converter-example will default to the first PDU in
# pdu_collection.c
#
# To use a specific PDU type, add -DPDU=<TypeName> to CFLAGS.
# NOTE: With -fprefix=ASN1_, type names are prefixed.
# Example: -DPDU=ASN1_YourTypeName
#
# See pdu_collection.c for the list of available PDU types.
```

Multiple ASN.1 Modules with Name Clashes

When generating code for multiple ASN.1 specifications that have type name clashes:

```
asn1c -fprefix=PKIX_ rfc3280.asn1
asn1c -fprefix=CMS_ rfc3852.asn1
```

This prevents symbol collisions in the global C namespace by ensuring types from different modules have different prefixes.

Integration with Existing Code

If your existing codebase has types that might conflict with generated ASN.1 types, use `-fprefix` to namespace the generated code:

```
asn1c -fprefix=ASN_ myprotocol.asn1
```

All generated types will have the `ASN_` prefix, avoiding conflicts with your existing type definitions.

Chapter 7

Abstract Syntax Notation: ASN.1

This chapter defines some basic ASN.1 concepts and describes several most widely used types. It is by no means an authoritative or complete reference. For more complete ASN.1 description, please refer to Olivier Dubuisson's book [Dub00] or the ASN.1 body of standards itself [ITU-T/ASN.1].

The Abstract Syntax Notation One is used to formally describe the data transmitted across the network. Two communicating parties may employ different formats of their native data types (e. g., different number of bits for the native integer type), thus it is important to have a way to describe the data in a manner which is independent from the particular machine's representation. The ASN.1 specifications are used to achieve the following:

- The specification expressed in the ASN.1 notation is a formal and precise way to communicate the structure of data to human readers;
- The ASN.1 specifications may be used as input for automatic compilers which produce the code for some target language (C, C++, Java, etc) to encode and decode the data according to some encoding formats. Several such encoding formats (called Transfer Encoding Rules) have been defined by the ASN.1 standard.

Consider the following example:

```
Rectangle ::= SEQUENCE {
    height  INTEGER,
    width   INTEGER
}
```

This ASN.1 specification describes a constructed type, *Rectangle*, containing two integer fields. This specification may tell the reader that there exists this kind of data structure and that some entity may be prepared to send or receive it. The question on *how* that entity is going

to send or receive the *encoded data* is outside the scope of ASN.1. For example, this data structure may be encoded according to some encoding rules and sent to the destination using the TCP protocol. The ASN.1 specifies several ways of encoding (or "serializing", or "marshaling") the data: BER, PER, XER and others, including CER and DER derivatives from BER.

The complete specification must be wrapped in a module, which looks like this:

```
RectangleModule1
    { iso org(3) dod(6) internet(1) private(4)
      enterprise(1) spelio(9363) software(1)
      asn1c(5) docs(2) rectangle(1) 1 }
    DEFINITIONS AUTOMATIC TAGS ::=
BEGIN

-- This is a comment which describes nothing.
Rectangle ::= SEQUENCE {
    height  INTEGER,        -- Height of the rectangle
    width   INTEGER         -- Width of the rectangle
}

END
```

The module header consists of module name (RectangleModule1), the module object identifier ({...}), a keyword "DEFINITIONS", a set of module flags (AUTOMATIC TAGS) and "::= BEGIN". The module ends with an "END" statement.

7.1 Some of the ASN.1 Basic Types

7.1.1 The BOOLEAN type

The BOOLEAN type models the simple binary TRUE/FALSE, YES/NO, ON/OFF or a similar kind of two-way choice.

7.1.2 The INTEGER type

The INTEGER type is a signed natural number type without any restrictions on its size. If the automatic checking on INTEGER value bounds are necessary, the subtype constraints must be used.

```
SimpleInteger ::= INTEGER

-- An integer with a very limited range
SmallPositiveInt ::= INTEGER (0..127)

-- Integer, negative
NegativeInt ::= INTEGER (MIN..0)
```

INTEGER constraints may use values that are larger than the traditional 32-bit or signed **`long`** range. The compiler preserves these bounds when generating constraint checking code, including unsigned 64-bit limits and values beyond 64 bits.

```
-- Fits uint32_t when generated with -finteger-native-type=uint32
Unsigned32 ::= INTEGER (0..4294967295)

-- Fits uint64_t when generated with -finteger-native-type=uint64
Unsigned64 ::= INTEGER (0..18446744073709551615)

-- Larger than uint64_t; generated as INTEGER_t, with the
-- arbitrary-precision bound still used for constraint checks.
Beyond64 ::= INTEGER (0..18446744073709551616)
```

By default, `asn1c` keeps the historical portable choice: constrained INTEGER values use **`long`** or **`unsigned long`** only when the range fits the conservative target **`long`** model, and otherwise use **`INTEGER_t`**. Use `-flong-size=64` when the generated code is meant for a 64-bit **`long`** target and you want that default policy to use the wider native range.

For APIs that require fixed-width integer members, use the `-finteger-native-type` option. Its *mode* may be `int32`, `uint32`, `int64`, `uint64`, or `auto`. The selected fixed-width type is used only when the ASN.1 range fits it exactly; otherwise **`INTEGER_t`** is generated. For example, compiling the ASN.1 above with `-finteger-native-type=uint64` generates native **`uint64_t`** storage for **`Unsigned64`**, while **`Beyond64`** remains **`INTEGER_t`** because its upper bound is larger than **`UINT64_MAX`**. Unsigned modes are used only for bounded ranges whose lower bound is known to be non-negative; open or negative-capable ranges fall back to **`INTEGER_t`**.

7.1.3 The ENUMERATED type

The ENUMERATED type is semantically equivalent to the INTEGER type with some integer values explicitly named.

```
FruitId ::= ENUMERATED { apple(1), orange(2) }

-- The numbers in braces are optional,
-- the enumeration can be performed
-- automatically by the compiler
ComputerOSType ::= ENUMERATED {
    FreeBSD,          -- acquires value 0
    Windows,          -- acquires value 1
    Solaris(5),       -- remains 5
    Linux,            -- becomes 6
    MacOS             -- becomes 7
}
```

7.1.4 The OCTET STRING type

This type models the sequence of 8-bit bytes. This may be used to transmit some opaque data or data serialized by other types of encoders (e. g., video file, photo picture, etc).

7.1.5 The OBJECT IDENTIFIER type

The OBJECT IDENTIFIER is used to represent the unique identifier of any object, starting from the very root of the registration tree. If your organization needs to uniquely identify something (a router, a room, a person, a standard, or whatever), you are encouraged to get your own identification subtree at http://www.iana.org/protocols/forms.htm.

For example, the very first ASN.1 module in this Chapter (RectangleModule1) has the following OBJECT IDENTIFIER: 1 3 6 1 4 1 9363 1 5 2 1 1.

```
ExampleOID ::= OBJECT IDENTIFIER

rectangleModule1-oid ExampleOID
  ::= { 1 3 6 1 4 1 9363 1 5 2 1 1 }

-- An identifier of the Internet.
internet-id OBJECT IDENTIFIER
  ::= { iso(1) identified-organization(3)
        dod(6) internet(1) }
```

As you see, names are optional.

7.1.6 The RELATIVE-OID type

The RELATIVE-OID type has the semantics of a subtree of an OBJECT IDENTIFIER. There may be no need to repeat the whole sequence of numbers from the root of the registration tree where the only thing of interest is some of the tree's subsequence.

```
this-document RELATIVE-OID ::= { docs(2) usage(1) }

this-example RELATIVE-OID ::= {
    this-document assorted-examples(0) this-example(1) }
```

7.2 Some of the ASN.1 String Types

7.2.1 The IA5String type

This is essentially the ASCII, with 128 character codes available (7 lower bits of an 8-bit byte).

7.2.2 The UTF8String type

This is the character string which encodes the full Unicode range (4 bytes) using multibyte character sequences.

7.2.3 The NumericString type

This type represents the character string with the alphabet consisting of numbers ("0" to "9") and a space.

7.2.4 The PrintableString type

The character string with the following alphabet: space, "**'**" (single quote), "**(**", "**)**", "**+**", "**,**" (comma), "**-**", "**.**", "**/**", digits ("0" to "9"), "**:**", "**=**", "**?**", upper-case and lower-case letters ("A" to "Z" and "a" to "z").

7.2.5 The VisibleString type

The character string with the alphabet which is more or less a subset of ASCII between the space and the "**~**" symbol (tilde).

Alternatively, the alphabet may be described as the PrintableString alphabet presented earlier, plus the following characters: "**!**", ""**"**", "**#**", "**$**", "**%**", "**&**", "*****", "**;**", "**<**", "**>**", "**[**", "****", "**]**", "**^**", "**_**", "**`**" (single left quote), "**{**", "**|**", "**}**", "**~**".

7.3 ASN.1 Constructed Types

7.3.1 The SEQUENCE type

This is an ordered collection of other simple or constructed types. The SEQUENCE constructed type resembles the C "struct" statement.

```
Address ::= SEQUENCE {
    -- The apartment number may be omitted
    apartmentNumber      NumericString OPTIONAL,
    streetName           PrintableString,
    cityName             PrintableString,
    stateName            PrintableString,
    -- This one may be omitted too
    zipNo                NumericString OPTIONAL
}
```

7.3.2 The SET type

This is a collection of other simple or constructed types. Ordering is not important. The data may arrive in the order which is different from the order of specification. Data is encoded in the order not necessarily corresponding to the order of specification.

7.3.3 The CHOICE type

This type is just a choice between the subtypes specified in it. The CHOICE type contains at most one of the subtypes specified, and it is always implicitly known which choice is being decoded or encoded. This one resembles the C "union" statement.

The following type defines a response code, which may be either an integer code or a boolean "true"/"false" code.

```
ResponseCode ::= CHOICE {
    intCode    INTEGER,
```

```
    boolCode    BOOLEAN
}
```

7.3.4 The SEQUENCE OF type

This one is the list (array) of simple or constructed types:

```
-- Example 1
ManyIntegers ::= SEQUENCE OF INTEGER

-- Example 2
ManyRectangles ::= SEQUENCE OF Rectangle

-- More complex example:
-- an array of structures defined in place.
ManyCircles ::= SEQUENCE OF SEQUENCE {
                            radius INTEGER
                            }
```

7.3.5 The SET OF type

The SET OF type models the bag of structures. It resembles the SEQUENCE OF type, but the order is not important. The elements may arrive in the order which is not necessarily the same as the in-memory order on the remote machines.

```
-- A set of structures defined elsewhere
SetOfApples :: SET OF Apple

-- Set of integers encoding the kind of a fruit
FruitBag ::= SET OF ENUMERATED { apple, orange }
```

Bibliography

[ASN1C] The Open Source ASN.1 Compiler. http://lionet.info/asn1c

[AONL] Online ASN.1 Compiler. http://lionet.info/asn1c/asn1c.cgi

[Dub00] Olivier Dubuisson — *ASN.1 Communication between heterogeneous systems* — Morgan Kaufmann Publishers, 2000. http://asn1.elibel.tm.fr/en/book/. ISBN:0-12-6333361-0.

[ITU-T/ASN.1] ITU-T Study Group 17 — Languages for Telecommunication Systems http://www.itu.int/ITU-T/studygroups/com17/languages/